

CSE 402: Numerical Methods

Complete Formula Sheet

Core Formulas + Derived Shortcuts + Exam Traps

Key Point / Exam Tip

How this sheet is built. **Part I** is every formula the lectures actually stated, in syllabus order. **Part II** is the part that wins marks under time pressure: formulas that are *implicit* in the slides but never written down (interval widths after n iterations, iteration counts, eigenvalue shortcuts, accuracy laws), the constants worth memorizing, and the traps that make an easy question look hard. Start with the *At a Glance* page.

Contents

At a Glance — The 40 Formulas Most Likely to Decide Your Grade	3
Part I — Core Formulas by Topic	6
1 Introduction, Approximations & Errors	6
2 Root Finding	6
3 Systems of Linear Equations	7
4 Eigenvalue Decomposition (Power Method)	7
5 Optimization	8
5.1 Golden Section Search	8
5.2 Gradient Descent	8
5.3 Newton's Method for Optimization	9
5.4 Foundations: Gradient & Stationary Points	9
5.5 Constrained Optimization (Lagrange Multipliers)	9
6 Regression	10
6.1 Least-Squares Linear Regression	10
6.2 Multiple Linear Regression	10
7 Interpolation	10
7.1 Lagrange Interpolation	10
7.2 Linear Interpolation	11
7.3 Newton's Divided-Difference Interpolation	11
7.4 Spline Interpolation	11
8 Simulation & Modeling	12
9 Random Number Generation & Monte Carlo	12
9.1 Random Number Generation	12

9.2 Monte Carlo Simulation	13
Part II — Derived Formulas, Quick Numbers & Traps	14
10 Errors & Root Finding — Beyond the Slides	14
10.1 Errors — Derived & Implicit Formulas	14
10.2 Root Finding — Derived & Implicit Formulas	16
11 Linear Systems & Eigenvalues — Beyond the Slides	18
11.1 Systems of Equations — Derived & Implicit Formulas	18
11.2 Eigenvalues — Derived & Implicit Formulas	20
12 Optimization — Beyond the Slides	22
12.1 Golden Section Search — Derived & Implicit Formulas	22
12.2 Gradient Descent — Derived & Implicit Formulas	24
12.3 Newton's Method for Optimization — Derived & Implicit Formulas	25
12.4 Stationary Points & Lagrange — Derived & Implicit Formulas	25
13 Regression & Interpolation — Beyond the Slides	27
13.1 Regression — Derived & Implicit Formulas	27
13.2 Interpolation — Derived & Implicit Formulas	31
14 Simulation, RNG & Monte Carlo — Beyond the Slides	35
14.1 Simulation & Modeling — Derived & Implicit Formulas	36
14.2 RNG & Monte Carlo — Derived & Implicit Formulas	38
Part III — Master Comparison Tables	42
15 Master Comparison Tables	42
15.1 Root-Finding Methods	42
15.2 Linear-System Methods	42
15.3 Optimization Methods	43
15.4 Interpolation Methods	43
15.5 RNG Statistical Tests	43
15.6 Convergence Orders — One Table	44

At a Glance — The 40 Formulas Most Likely to Decide Your Grade

Key Point / Exam Tip

If you memorize nothing else, memorize this page. Entries marked ★ are *derived* results that do **not** appear on the slides but turn multi-minute calculations into one line — exactly the kind that has shown up in past MCQs.

Situation	Formula
Errors	
Relative approx. error	$\epsilon_a = \frac{\text{present} - \text{previous}}{\text{present}}$
Digits guaranteed	$ \epsilon_a \leq 0.5 \times 10^{2-m\%}$
★ Invert it	$m = \lfloor 2 - \log_{10}(2 \epsilon_a) \rfloor \quad (\epsilon_a \text{ in } \%)$
Correct to M digits	$ x - \hat{x} \leq \frac{1}{2} \times 10^{n-M+1}, \quad n = \lfloor \log_{10} x \rfloor$
★ Error trade-off	total error = round-off + truncation is U-shaped in h ; shrinking h forever makes it worse
Root Finding	
Bisection step	$x_m = \frac{x_\ell + x_u}{2}$, requires $f(x_\ell)f(x_u) < 0$
★ Width after n	$L_n = \frac{b-a}{2^n}$, error bound $\frac{b-a}{2^{n+1}}$
★ Iterations needed	$n \geq \log_2 \frac{b-a}{\epsilon} = \frac{\ln((b-a)/\epsilon)}{\ln 2}$
False position	$x_r = \frac{x_U f(x_L) - x_L f(x_U)}{f(x_L) - f(x_U)}$
Newton–Raphson	$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)}$
★ NR error law	$e_{i+1} \approx \frac{f''(r)}{2f'(r)} e_i^2$ (quadratic: digits double)
★ NR on multiple root	only linear , rate $1 - \frac{1}{k}$ for multiplicity k ; fix: $x - k f/f'$
Linear Systems	
Multiplier	$m_{ik} = a_{ik}^{(k-1)} / a_{kk}^{(k-1)}$
★ Determinant	$\det A = (-1)^{\#\text{swaps}} \prod_i u_{ii}$ (since $\det L = 1$)
LU solve	$LZ = C$ (forward), then $UX = Z$ (back)
★ Cost	Gauss $\approx \frac{2}{3}n^3$; Gauss–Jordan $\approx n^3$; A^{-1} : LU $\sim n^3$ vs Gauss $\sim n^4$, ratio $\approx n/4$
Eigenvalues	
Definition	$A\mathbf{v} = \lambda\mathbf{v} \Rightarrow \det(A - \lambda I) = 0$
Decomposition	$A = V\Lambda V^{-1}$, $A^k = V\Lambda^k V^{-1}$

Situation	Formula
★ Instant checks	$\text{tr } A = \sum \lambda_i, \quad \det A = \prod \lambda_i$
★ Spectrum map	$A^k \rightarrow \lambda^k; \quad A^{-1} \rightarrow 1/\lambda; \quad A + cI \rightarrow \lambda + c; \quad cA \rightarrow c\lambda$ (<i>eigenvectors</i> unchanged in every case)
Power method	$\mathbf{x}_{k+1} = \frac{A\mathbf{x}_k}{\text{largest-magnitude entry}}, \quad \lambda_1 \approx \text{that entry}$
★ Convergence rate	unwanted part decays as $ \lambda_2/\lambda_1 ^k$
Deflation	$A_2 = A - \lambda_1 \hat{\mathbf{v}}_1 \hat{\mathbf{v}}_1^\top$ (spectrum $\{0, \lambda_2, \dots\}$)

Optimization

Golden ratio	$R = \frac{\sqrt{5} - 1}{2} = 0.618034, \quad d = R(x_u - x_L)$
★ GSS width after n	$\boxed{L_n = R^n L_0} \quad R^3 = 0.236068, \quad R^5 = 0.090170$
★ GSS iterations needed	$n \geq \frac{\ln(\varepsilon/L_0)}{\ln R}$
★ GSS identities	$R^2 = 1 - R = 0.381966, \quad 1/R = 1 + R = 1.618034$
★ GSS evaluations	2 interior evals on iter. 1, then 1 each $\Rightarrow n + 1$ after n iters
GD update	$w_j \leftarrow w_j - \alpha \partial J / \partial w_j$
★ GD on $J = a(x - x^*)^2$	$e_{k+1} = (1 - 2a\alpha)e_k$; converges iff $0 < \alpha < 1/a$; exact in one step at $\alpha = 1/(2a)$
Newton (opt.)	$x_{t+1} = x_t - \frac{f'(x_t)}{f''(x_t)}, \quad \text{vector: } x_t - H^{-1}g$
★ Newton on a quadratic	converges in exactly one step from any start
★ 2-D classification	$\det H > 0, f_{xx} > 0 \Rightarrow \text{min}; \det H > 0, f_{xx} < 0 \Rightarrow \text{max};$ $\det H < 0 \Rightarrow \text{saddle}$
Lagrange	$\nabla J + \lambda \nabla C = 0, \quad C(\mathbf{x}) = 0$

Regression

Slope / intercept	$m = \frac{n \sum xy - \sum x \sum y}{n \sum x^2 - (\sum x)^2}, \quad b = \frac{\sum y - m \sum x}{n}$
★ Centroid property	the fit always passes through $(\bar{x}, \bar{y})$; $\sum e_i = 0$
Correlation	$r = \frac{n \sum xy - \sum x \sum y}{\sqrt{[n \sum x^2 - (\sum x)^2][n \sum y^2 - (\sum y)^2]}}$
★ Shortcut	$m = r \frac{s_y}{s_x}, \quad r^2 = 1 - \frac{SSE}{SST}$
MLR cost & update	$J = \frac{1}{2m} \sum (\hat{y}_i - y_i)^2, \quad w_j \leftarrow w_j - \frac{\alpha}{m} \sum (\hat{y}_i - y_i) x_{ij}$

Interpolation

Lagrange	$f(x) = \sum_j y_j \prod_{k \neq j} \frac{x - x_k}{x_j - x_k}$
Newton form	$P_n = a_0 + a_1(x - x_0) + \dots, \quad a_k = f[x_0 \dots x_k]$

Situation	Formula
Divided difference	$f[x_0..x_{n+1}] = \frac{f[x_1..x_{n+1}] - f[x_0..x_n]}{x_{n+1} - x_0}$
★ D.d. $\leftrightarrow$ derivative	$f[x_0..x_n] = \frac{f^{(n)}(\xi)}{n!} \Rightarrow$ for a degree- n polynomial the n -th d.d. is the leading coefficient and the $(n+1)$ -th is 0
★ Same polynomial	Lagrange and Newton give identical values — only the form differs
Spline counts	linear $2n$; quadratic $3n = 2n + (n-1) + 1$; cubic $4n = 2n + (n-1) + (n-1) + 2$ ($n = k - 1$ for k points)
Simulation	
Average delay	$\hat{d}(n) = \frac{1}{n} \sum D_i$ (discrete-time average)
Time averages	$\hat{q}(n) = \frac{\int_0^T Q(t)dt}{T}$, $\hat{u}(n) = \frac{\int_0^T B(t)dt}{T}$
Area rule	new area = old state $\times$ (clock – time of last event)
★ Little's Law	$L = \lambda W$, $L_q = \lambda W_q$
★ Utilization	$\rho = \lambda/\mu$; stable only if $\lambda < \mu$
RNG & Monte Carlo	
LCG	$X_{i+1} = (aX_i + c) \bmod m$, $R_i = X_i/m$
$U(0,1)$ moments	$E[R] = \frac{1}{2}$, $E[R^2] = \frac{1}{3}$, $\text{Var} = \frac{1}{12} = 0.0833$
Multiple testing	$P(\geq 1 \text{ false reject}) = 1 - (1 - \alpha)^k$
Autocorrelation	$\hat{\rho}_{i\ell} = \frac{1}{M+1} \sum R_{i+k\ell} R_{i+(k+1)\ell} - 0.25$, $\sigma = \frac{\sqrt{13M+7}}{12(M+1)}$
MC estimator	$\bar{Y}(n) = (b-a) \frac{1}{n} \sum g(X_i)$, $\hat{\pi} = 4h/n$
★ MC accuracy law	error $\propto \frac{\sigma}{\sqrt{n}} \Rightarrow 100\times$ the samples for $10\times$ the accuracy

Part I — Core Formulas by Topic

Everything the lectures stated, in syllabus order

1 Introduction, Approximations & Errors

True error and relative true error:

$$E_t = \text{True} - \text{Approx.}, \quad \epsilon_t = \frac{E_t}{\text{True value}}$$

Approximate error and relative approximate error (present – previous):

$$E_a = \text{Present} - \text{Previous}, \quad \epsilon_a = \frac{E_a}{\text{Present}}$$

Stopping rules:

$$|\epsilon_a| \leq \epsilon_s \quad (\text{tolerance})$$

$$|\epsilon_a| \leq 0.5 \times 10^{2-m}\% \quad (\text{certifies } m \text{ correct sig. digits})$$

Normalized scientific notation: $|x| = m \times 10^n$, $1 \leq m < 10$, $n = \lfloor \log_{10} |x| \rfloor$. M -th significant digit sits at place 10^{n-M+1} .

“Correct to M sig. digits” (absolute-error form):

$$|x - \hat{x}| \leq \frac{1}{2} \times 10^{n-M+1}$$

Maclaurin series:

$$f(x) = \sum_{n=0}^{\infty} \frac{f^{(n)}(0)}{n!} x^n$$

Error sources: modelling error, data/measurement error, programming mistakes (avoidable); **round-off error** (finite precision) and **truncation error** (cutting short an infinite/exact procedure) — inherent, unavoidable.

2 Root Finding

Bisection. Requires $f(x_\ell)f(x_u) < 0$ (Intermediate Value Theorem: guarantees ≥ 1 root, never exactly one).

$$x_m = \frac{x_\ell + x_u}{2}$$

Update: $f(x_\ell)f(x_m) < 0 \Rightarrow x_u = x_m$; $f(x_\ell)f(x_m) > 0 \Rightarrow x_\ell = x_m$.

False Position. From similar triangles on the secant:

$$x_r = \frac{x_U f(x_L) - x_L f(x_U)}{f(x_L) - f(x_U)} = x_U - \frac{f(x_U)(x_U - x_L)}{f(x_U) - f(x_L)}$$

Update rule identical in form to bisection, using x_r in place of x_m .

Newton–Raphson. From the tangent-slope relation $f'(x_i) = \frac{f(x_i)}{x_i - x_{i+1}}$:

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)}$$

Stopping error (all three methods, with x^{new} = latest estimate):

$$|\epsilon_a| = \left| \frac{x^{new} - x^{old}}{x^{new}} \right| \times 100\%$$

Convergence: Bisection/False-Position $\rightarrow$ guaranteed, linear. Newton–Raphson $\rightarrow$ not guaranteed, quadratic (correct digits double each step) when it converges.

NR failure modes: $f'(x_i) = 0$ (division by zero), inflection points nearby, oscillation near local max/min (no real root), root jumping (multiple/periodic roots).

3 Systems of Linear Equations

Gauss elimination. Multiplier and update at step k :

$$m_{ik} = \frac{a_{ik}^{(k-1)}}{a_{kk}^{(k-1)}}, \quad a_{ij}^{(k)} = a_{ij}^{(k-1)} - m_{ik} a_{kj}^{(k-1)}$$

Back substitution:

$$x_n = \frac{b_n^{(n-1)}}{a_{nn}^{(n-1)}}, \quad x_i = \frac{b_i^{(i-1)} - \sum_{j=i+1}^n a_{ij}^{(i-1)} x_j}{a_{ii}^{(i-1)}}$$

Partial pivoting: swap in $\max_i |a_{ik}|$ as pivot each step k — fixes zero-pivot and round-off blowup.

Determinant: row add/sub $\Rightarrow$ det unchanged; triangular $\Rightarrow$ det = $\prod a_{ii}$; row swap $\Rightarrow$ det flips sign.

Gauss–Jordan: normalize each pivot row to 1, eliminate that column from *every* other row (above & below) $\Rightarrow$ coefficient matrix $\rightarrow I$; RHS = solution (no back-sub).

LU decomposition: $A = LU$; U = result of naive forward elimination; L = unit-lower-triangular with m_{ij} at position (i, j) .

$$LZ = C \text{ (forward sub.)}, \quad UX = Z \text{ (back sub.)}$$

$$z_1 = c_1, \quad z_i = c_i - \sum_{j < i} \ell_{ij} z_j$$

$$x_n = \frac{z_n}{u_{nn}}, \quad x_i = \frac{z_i - \sum_{j > i} u_{ij} x_j}{u_{ii}}$$

Cost: single solve $C_T|_{LU} = C_T|_{GE}$ (identical); for A^{-1} (n RHS's), $LU \sim n^3$ total vs. Gauss $\sim n^4$ — ratio $\approx n/4$.

4 Eigenvalue Decomposition (Power Method)

Defining/characteristic equation:

$$A\mathbf{v} = \lambda\mathbf{v} \Rightarrow (A - \lambda I)\mathbf{v} = 0 \Rightarrow \det(A - \lambda I) = 0$$

Eigen decomposition ($V = [\mathbf{v}_1 \cdots \mathbf{v}_n]$, $\Lambda = \text{diag}(\lambda_i)$):

$$AV = V\Lambda \Rightarrow A = V\Lambda V^{-1}, \quad A^k = V\Lambda^k V^{-1}$$

Power method ($\mathbf{x} = \sum c_i \mathbf{v}_i$, needs $|\lambda_1| > |\lambda_2| \geq \cdots$):

$$A^k \mathbf{x} = \lambda_1^k \left[c_1 \mathbf{v}_1 + \sum_{i \geq 2} c_i \left(\frac{\lambda_i}{\lambda_1} \right)^k \mathbf{v}_i \right] \rightarrow \lambda_1^k c_1 \mathbf{v}_1$$

Iteration:

$$\mathbf{y}_{k+1} = A\mathbf{x}_k, \quad \mathbf{x}_{k+1} = \frac{\mathbf{y}_{k+1}}{\text{largest-magnitude entry}}$$

$$\lambda_1 \approx \text{that largest-magnitude entry}$$

$$|\varepsilon_a| = \left| \frac{\lambda_k - \lambda_{k-1}}{\lambda_k} \right| \times 100\%$$

Inverse power method (smallest eigenvalue): $A\mathbf{v} = \lambda\mathbf{v} \Rightarrow A^{-1}\mathbf{v} = \frac{1}{\lambda}\mathbf{v}$; run power method on A^{-1} (solve $A\mathbf{y} = \mathbf{x}$ via LU), $\lambda_{\min} \approx 1/(\text{normalizing factor})$.

Deflation (intermediate eigenvalues, symmetric A): $\hat{\mathbf{v}}_1 = \mathbf{v}_1 / \sqrt{\mathbf{v}_1^\top \mathbf{v}_1}$,

$$A_2 = A - \lambda_1 \hat{\mathbf{v}}_1 \hat{\mathbf{v}}_1^\top$$

power method on $A_2 \rightarrow \lambda_2$; repeat $A_3 = A_2 - \lambda_2 \hat{\mathbf{v}}_2 \hat{\mathbf{v}}_2^\top$, etc.

5 Optimization

5.1 Golden Section Search

Golden ratio and interior-point placement:

$$R = \frac{\sqrt{5} - 1}{2} \approx 0.618, \quad d = R(x_u - x_L)$$

$$x_1 = x_u - d, \quad x_2 = x_L + d$$

Decision rule (maximization): $f_1 > f_2 \Rightarrow \text{max is left } (x_u \leftarrow x_2)$; $f_1 < f_2 \Rightarrow \text{max is right } (x_L \leftarrow x_1)$. Reuse the surviving interior point; only *one* new function value is computed per iteration after the first.

$$x_{opt} = \frac{x_u + x_L}{2}, \quad f_{opt} = f(x_{opt})$$

Stopping criterion:

$$\left| \frac{2 - R}{R} \right| \cdot \left| \frac{x_u - x_L}{x_u + x_L} \right| \text{ or simply } \left| \frac{x_u - x_L}{x_u + x_L} \right| \times 2 < \varepsilon_s$$

5.2 Gradient Descent

Single variable:

$$\text{step} = \frac{d(\text{loss})}{d(\text{param})} \times \alpha, \quad \text{param}_{new} = \text{param}_{old} - \text{step}$$

General parameter w_j ($\alpha = \text{learning rate}$):

$$w_j \leftarrow w_j - \alpha \frac{\partial J}{\partial w_j}$$

SSR loss and its gradients (simple linear regression $\hat{y} = b + mx$, residual $r_i = y_i - (b + mx_i)$):

$$SSR = \sum_{i=1}^n (y_i - b - mx_i)^2$$

$$\frac{\partial SSR}{\partial b} = -2 \sum_{i=1}^n (y_i - b - mx_i)$$

$$\frac{\partial SSR}{\partial m} = -2 \sum_{i=1}^n x_i (y_i - b - mx_i)$$

Stop when: step size ≈ 0 **or** max iterations reached.

5.3 Newton's Method for Optimization

Root finding: $x_{t+1} = x_t - \frac{f(x_t)}{f'(x_t)}$

Optimization: $x_{t+1} = x_t - \frac{f'(x_t)}{f''(x_t)}$

Vector / multivariate form ($g = \nabla f$, $H = \nabla^2 f$):

$$x_{t+1} = x_t - H^{-1}g$$

Damped: $x_{t+1} = x_t - t H^{-1}g$, $t > 0$. Practical: solve $Hy = g$ for y , then $x_{t+1} = x_t - y$ (avoid explicit inversion).

Second-order minimum condition: $\nabla^2 q = H$ must be PSD (all eigenvalues ≥ 0).

Convex $f \Rightarrow H$ PSD. Quadratic convergence: correct digits $\approx$ double each step.

5.4 Foundations: Gradient & Stationary Points

1-D: solve $f'(x) = 0$; classify with $f''(x)$: $f'' > 0$ min, $f'' < 0$ max, sign change (or $f'' = 0$) $\Rightarrow$ saddle/inflection. n -D gradient vector:

$$\nabla J(\mathbf{x}) = \left(\frac{\partial J}{\partial x_1}, \frac{\partial J}{\partial x_2}, \dots, \frac{\partial J}{\partial x_n} \right)^\top$$

Basis of optimization: (1) set up objective function; (2) solve $\nabla J = 0$.

5.5 Constrained Optimization (Lagrange Multipliers)

Problem: minimize/maximize $J(\mathbf{x})$ subject to $C(\mathbf{x}) = 0$. Lagrangian:

$$\Lambda(\mathbf{x}, \lambda) = J(\mathbf{x}) + \lambda C(\mathbf{x})$$

Stationarity (parallel-gradient) condition:

$$\nabla J(\mathbf{x}) + \lambda \nabla C(\mathbf{x}) = 0 \iff \nabla J = -\lambda \nabla C$$

Bordered Hessian (n variables):

$$H(\Lambda) = \begin{pmatrix} 0 & \partial C / \partial x_1 & \cdots & \partial C / \partial x_n \\ \partial C / \partial x_1 & \partial^2 \Lambda / \partial x_1^2 & \cdots & \partial^2 \Lambda / \partial x_1 \partial x_n \\ \vdots & \vdots & \ddots & \vdots \\ \partial C / \partial x_n & \partial^2 \Lambda / \partial x_1 \partial x_n & \cdots & \partial^2 \Lambda / \partial x_n^2 \end{pmatrix}$$

used to classify the stationary point as min / max / saddle. If $C(\mathbf{x}) = g$ (not 0), first rewrite as $C(\mathbf{x}) - g = 0$.

6 Regression

6.1 Least-Squares Linear Regression

Model $\hat{y} = mx + b$, error = $y_i - (mx_i + b)$:

$$SSE = \sum_{i=1}^n (y_i - mx_i - b)^2$$

Normal-equation results:

$$b = \frac{\sum y - m \sum x}{n}, \quad m = \frac{n \sum xy - \sum x \sum y}{n \sum x^2 - (\sum x)^2}$$

Correlation coefficient and coefficient of determination:

$$r = \frac{n \sum xy - \sum x \sum y}{\sqrt{[n \sum x^2 - (\sum x)^2][n \sum y^2 - (\sum y)^2]}}$$

r^2 = fraction of variance explained. Running sums needed: $\sum x$, $\sum y$, $\sum x^2$, $\sum y^2$, $\sum xy$.

6.2 Multiple Linear Regression

Hyperplane model:

$$\hat{y} = w_0 + w_1x_1 + w_2x_2 + \cdots + w_nx_n$$

Cost function (Mean Squared Error, $m = \#$ samples):

$$J(\mathbf{w}) = \frac{1}{2m} \sum_{i=1}^m (\hat{y}_i - y_i)^2$$

Gradient of the cost (with $\hat{y}_i = \sum_k w_k x_{ik}$, $x_{i0} = 1$):

$$\frac{\partial J}{\partial w_j} = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i) x_{ij}$$

Gradient descent update:

$$w_j \leftarrow w_j - \alpha \frac{\partial J}{\partial w_j} = w_j - \frac{\alpha}{m} \sum_{i=1}^m (\hat{y}_i - y_i) x_{ij}$$

Five assumptions: Linearity, Independence, Homoscedasticity (constant error variance), Normality (errors $\sim \mathcal{N}$), No multicollinearity.

7 Interpolation

7.1 Lagrange Interpolation

$(n + 1)$ points $\Rightarrow$ at most an n -th order polynomial:

$$f(x) = \sum_{j=0}^n y_j \prod_{\substack{k=0 \\ k \neq j}}^n \frac{x - x_k}{x_j - x_k}$$

7.2 Linear Interpolation

Bracketing points $(x_2, y_2), (x_3, y_3)$, query x_m :

$$\frac{y_3 - y_2}{x_3 - x_2} = \frac{y_m - y_2}{x_m - x_2}$$

$$\implies y_m = y_2 + \frac{y_3 - y_2}{x_3 - x_2}(x_m - x_2)$$

Always use the two tabulated points that bracket the query x .

7.3 Newton's Divided-Difference Interpolation

$$P_n(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + \dots$$

$$a_0 = y_0, \quad a_1 = f[x_1, x_0] = \frac{y_1 - y_0}{x_1 - x_0}$$

$$a_2 = f[x_2, x_1, x_0], \quad \dots$$

Master recurrence:

$$f[x_0, x_1, \dots, x_{n+1}] = \frac{f[x_1, \dots, x_{n+1}] - f[x_0, \dots, x_n]}{x_{n+1} - x_0}$$

Divided-difference table (top diagonal = coefficients $a_0, a_1, a_2, \dots$):

x_0	y_0			
		$f[x_0, x_1]$		
x_1	y_1		$f[x_0, x_1, x_2]$	
		$f[x_1, x_2]$		$f[x_0, x_1, x_2, x_3]$
x_2	y_2		$f[x_1, x_2, x_3]$	
		$f[x_2, x_3]$		
x_3	y_3			

Advantage over Lagrange: adding a new point only appends one row/coefficient (no recomputation).

7.4 Spline Interpolation

Order	Piece	Unknowns
Linear	$P_i = m_i x + c_i$	$2n$
Quadratic	$P_i = a_i x^2 + b_i x + c_i$	$3n$
Cubic	$P_i = a_i x^3 + b_i x^2 + c_i x + d_i$	$4n$

Quadratic equation count: $3n = \underbrace{2n}_{\text{through pts}} + \underbrace{(n-1)}_{\text{match } P'} + \underbrace{1}_{P_1''(x_1)=0}$

Cubic equation count: $4n = \underbrace{2n}_{\text{through pts}} + \underbrace{(n-1)}_{\text{match } P'} + \underbrace{(n-1)}_{\text{match } P''} + \underbrace{2}_{\text{boundary}}$

Natural cubic: $P_1''(x_{\text{first}}) = 0$, $P_n''(x_{\text{last}}) = 0$. Clamped cubic: end slopes dy/dx given. Runge's phenomenon $\Rightarrow$ motivation for splines (local, low-order pieces) over one global high-order polynomial.

8 Simulation & Modeling

System components: Entity, Attribute, Activity, Event, State variable. **Model classification:** Static/Dynamic; Deterministic/Stochastic; Discrete/Continuous. DES = Dynamic + Stochastic + Discrete.

Queueing notation: A_i = interarrival time, S_i = service time, D_i = delay in queue.

$$\text{Arrival time of customer } i = \sum_{j=1}^i A_j$$

$$\text{Total time in system} = D_i + S_i$$

Average delay (discrete-time statistic):

$$\hat{d}(n) = \frac{1}{n} \sum_{i=1}^n D_i$$

Time-average # in queue & utilization (continuous-time, time-weighted):

$$\hat{q}(n) = \frac{\int_0^{T(n)} Q(t) dt}{T(n)}$$

$$\hat{u}(n) = \frac{\int_0^{T(n)} B(t) dt}{T(n)} = \frac{\text{total busy time}}{T(n)}$$

Area accumulation rule (use the *old* value before updating state):

$$\text{new area} = (\text{old } Q(t) \text{ or } B(t)) \times (\text{clock} - \text{time of last event})$$

Event order (critical): (1) update area accumulators (old values) → (2) update state → (3) update time-of-last-event → (4) update event list.

V&V: Verification = “built the model right?” (code vs. conceptual model). Validation = “built the right model?” (model vs. reality). Calibration = iterative compare-revise loop driving validation. Input-output transformation: $f(X, D) = Y$ (X =inputs, D =decision vars, Y =outputs).

10 Steps in a Simulation Study: (1) Formulate problem & plan; (2) Collect data & define model; (3) Assumptions valid? (else → step 2); (4) Construct & verify program; (5) Pilot runs; (6) Model valid? (else → step 2); (7) Design experiments; (8) Production runs; (9) Analyze output; (10) Document, present, use.

Naylor–Finger validation: (1) Face validity; (2) Validate assumptions; (3) Validate I/O transformations.

9 Random Number Generation & Monte Carlo

9.1 Random Number Generation

Two Sacred Properties: **Uniformity & Independence** ($R_i \sim U(0, 1)$ i.i.d.).

$$E(R) = \frac{1}{2}, \quad E(R^2) = \frac{1}{3}, \quad \text{Var}(R) = \frac{1}{12}$$

LCG recurrence ($c \neq 0$: mixed; $c = 0$: multiplicative):

$$X_{i+1} = (aX_i + c) \bmod m, \quad R_i = \frac{X_i}{m}$$

Max. period conditions:

- $m = 2^b$, $c \neq 0$: $P = m$ if $\gcd(c, m) = 1$ and $a = 1 + 4k$.
- $m = 2^b$, $c = 0$: $P = m/4$ if X_0 odd, $a = 3 + 8k$ or $5 + 8k$.
- m prime, $c = 0$: $P = m - 1$ if smallest k with $m \mid (a^k - 1)$ is $k = m - 1$.

Multiple testing: $P(\geq 1 \text{ false reject}) = 1 - (1 - \alpha)^k$.

K-S test: $D = \max(D^+, D^-)$,

$$D_i^+ = \frac{i}{N} - R_{(i)}, \quad D_i^- = R_{(i)} - \frac{i-1}{N}$$

reject H_0 if $D > D_\alpha$.

Chi-square:

$$\chi_0^2 = \sum_{i=1}^n \frac{(O_i - E_i)^2}{E_i}, \quad E_i = \frac{N}{n}$$

reject if $\chi_0^2 > \chi_{\alpha, n-1}^2$.

Autocorrelation (lag ℓ , start i , M = largest int. with $i + (M+1)\ell \leq N$):

$$\hat{\rho}_{i\ell} = \frac{1}{M+1} \sum_{k=0}^M R_{i+k\ell} R_{i+(k+1)\ell} - 0.25$$

$$\sigma_{\hat{\rho}_{i\ell}} = \frac{\sqrt{13M+7}}{12(M+1)}, \quad Z_0 = \frac{\hat{\rho}_{i\ell}}{\sigma_{\hat{\rho}_{i\ell}}}$$

reject H_0 (dependence) if $|Z_0| > 1.96$ ($\alpha = 0.05$).

9.2 Monte Carlo Simulation

Integration as expectation: $X \sim U(a, b)$, $Y = (b-a)g(X) \Rightarrow E(Y) = \int_a^b g(x) dx$.

Sample-mean estimator:

$$\bar{Y}(n) = (b-a) \cdot \frac{1}{n} \sum_{i=1}^n g(X_i) \approx I$$

Monte Carlo estimate of π (quarter circle in unit square, h = hits with $X_i^2 + Y_i^2 \leq 1$):

$$\hat{\pi} = \frac{4h}{n}$$

Estimator $\bar{Y}(n)$ is itself a random variable (error need not shrink monotonically in n); justified by the Law of Large Numbers.

Part II — Derived Formulas, Quick Numbers & Traps

The shortcuts the slides never wrote down

10 Errors & Root Finding — Beyond the Slides

10.1 Errors — Derived & Implicit Formulas

Derived & Implicit Formulas — not on the slides, but exam-critical**(E1) Inverting the significant-digit criterion — the most useful hidden formula here.**

The slides give one direction only: $|\epsilon_a| \leq 0.5 \times 10^{2-m}\%$ certifies m digits. Solve it for m :

$$|\epsilon_a| \leq 0.5 \times 10^{2-m} \implies \log_{10}(2|\epsilon_a|) \leq 2 - m$$

$$m = \lfloor 2 - \log_{10}(2|\epsilon_a|) \rfloor \quad (|\epsilon_a| \text{ expressed in } \%)$$

Now an MCQ like “ $|\epsilon_a| = 0.04\%$, how many digits are trusted?” is one line: $\lfloor 2 - \log_{10} 0.08 \rfloor = \lfloor 3.097 \rfloor = 3$.

(E2) The two error criteria are the same statement. Relative form $\Leftrightarrow$ absolute form:

$$\left| \frac{x - \hat{x}}{x} \right| \leq \frac{1}{2} \times 10^{-M} \iff |x - \hat{x}| \leq \frac{1}{2} \times 10^{n-M+1}, \quad n = \lfloor \log_{10} |x| \rfloor.$$

The right-hand side is exactly “half a unit in the place of the M -th significant digit” — i.e. the error you make by correctly *rounding* to M figures.

(E3) Taylor remainder and the order of a truncation error.

$$f(x+h) = \sum_{k=0}^n \frac{f^{(k)}(x)}{k!} h^k + \underbrace{\frac{f^{(n+1)}(\xi)}{(n+1)!} h^{n+1}}_{R_n}, \quad R_n = O(h^{n+1}).$$

Consequence for derivatives:

$$\underbrace{\frac{f(x+h) - f(x)}{h}}_{\text{forward difference}} = f'(x) + O(h), \quad \underbrace{\frac{f(x+h) - f(x-h)}{2h}}_{\text{central difference}} = f'(x) + O(h^2).$$

So halving h halves a forward-difference error but *quarters* a central-difference error.

(E4) The round-off / truncation trade-off (U-shaped total error). Truncation error falls like h ; round-off in the subtraction $f(x+h) - f(x)$ grows like $\varepsilon_{\text{mach}}/h$. Total:

$$E_{\text{tot}}(h) \approx \underbrace{C_1 h}_{\text{truncation}} + \underbrace{\frac{C_2 \varepsilon_{\text{mach}}}{h}}_{\text{round-off}}$$

which is minimized at $h^* \propto \sqrt{\varepsilon_{\text{mach}}}$. **Shrinking h past h^* makes the answer worse**, not better — the single most counter-intuitive fact in this topic.

(E5) Error propagation (implicit whenever a question chains computations).

$$\text{sums/differences: absolute errors add} \quad |E_{x \pm y}| \leq |E_x| + |E_y|$$

$$\text{products/quotients: relative errors add} \quad |\epsilon_{xy}| \approx |\epsilon_x| + |\epsilon_y|$$

This is why subtracting two nearly equal numbers is catastrophic: the absolute error stays the same while the result shrinks, so the *relative* error explodes.

Quick Numbers (memorize)

Digit table — read it straight off. $m = \lfloor 2 - \log_{10}(2|\epsilon_a|) \rfloor$, $|\epsilon_a|$ in %:

$ \epsilon_a $	5%	1%	0.5%	0.1%	0.05%	0.005%
digits m	1	1	2	2	3	4

Threshold form (equivalent, often faster): $m = 1 \Rightarrow 5\%$; $m = 2 \Rightarrow 0.5\%$; $m = 3 \Rightarrow 0.05\%$; $m = 4 \Rightarrow 0.005\%$. Each extra digit costs a factor of 10.

Worth knowing cold. $\frac{1}{3} - 0.333333 = 3.3333 \times 10^{-7}$ (6-digit round-off). $\int_3^9 x^2 dx = 234$; 2 rectangles give 135 (error 99), 4 give 182.25 (error 51.75). $e^{1.2}$ series needs $n = 7$ terms for 2 digits ($|\epsilon_a| = 0.12494\%$). Trunnion: wrong model $-0.01504''$ vs correct $-0.013689''$ against $0.015''$ required.

Exam Traps

- ϵ_a is **undefined on the first iteration** — there is no previous estimate. Tables always start with a dash.
- “More iterations” does not mean “more accuracy” once round-off dominates (E4).
- Decimal places $\neq$ significant figures. 0.00640 has **5** decimal places but **3** significant figures.
- E_t uses the *true* value, E_a uses the *previous* iterate. A question giving you only successive iterates cannot be asking for E_t .
- The sign of E_t matters: $E_t < 0$ means the approximation *overshot*. Many MCQs list both + and – options.
- Central difference is $O(h^2)$ but needs *two* extra evaluations; forward is $O(h)$ with one. Accuracy and cost move in opposite directions.
- A “correct” calculation on a wrong model is still wrong (the trunnion). Modelling error is not reduced by more precision.
- $0.5 \times 10^{2-m}\%$ already contains the %; do not divide by 100 twice.

10.2 Root Finding — Derived & Implicit Formulas

Derived & Implicit Formulas — not on the slides, but exam-critical

(R1) Bisection interval width and error bound after n iterations. Each iteration halves the bracket exactly, so

$$L_n = \frac{b-a}{2^n} \quad \text{and} \quad |x^* - x_m^{(n)}| \leq \frac{b-a}{2^{n+1}}$$

(the estimate is the midpoint, so the error is at most *half* the bracket).

(R2) Iterations required, known before you start. Setting $L_n \leq \epsilon$:

$$n \geq \log_2 \frac{b-a}{\epsilon} = \frac{\ln((b-a)/\epsilon)}{\ln 2}$$

and round *up*. This is the property the slides call “predictable error.” Note what is absent: f itself. **Bisection’s iteration count depends only on the bracket and the tolerance, never on the function.**

(R3) Order of convergence. With $e_k = |x_k - x^*|$, a method has order p if $e_{k+1} \approx C e_k^p$:

Method	p	C	Meaning
Bisection	1	$1/2$	one bit per iteration
False position	1	—	linear; slower when an endpoint sticks
Newton–Raphson	2	$\left \frac{f''(x^*)}{2f'(x^*)} \right $	correct digits <i>double</i>

(R4) Newton’s error law, derived. Taylor-expand f about the root x^* and substitute the Newton step:

$$e_{k+1} \approx \frac{f''(x^*)}{2f'(x^*)} e_k^2$$

This is what “quadratic convergence” literally means: squaring the error each step, hence doubling the number of correct digits. It also shows Newton breaks down when $f'(x^*) \rightarrow 0$.

(R5) Newton at a repeated root — the big trap. If x^* has multiplicity $k > 1$, then $f'(x^*) = 0$ too and (R4) fails. Newton degrades to **linear** convergence with rate $1 - \frac{1}{k}$:

$$e_{k+1} \approx \left(1 - \frac{1}{k}\right) e_k.$$

For $f(x) = x^2$ (double root at 0) the iteration collapses to $x_{k+1} = x_k - \frac{x_k^2}{2x_k} = \frac{x_k}{2}$ — exactly halving, i.e. *no faster than bisection*. The repair is the **modified Newton** step

$$x_{k+1} = x_k - k \frac{f(x_k)}{f'(x_k)},$$

which restores quadratic convergence.

(R6) Two classical Newton specializations worth recognizing on sight.

$$f(x) = x^2 - a \Rightarrow x_{k+1} = \frac{1}{2} \left(x_k + \frac{a}{x_k} \right) \quad (\text{square-root / “Babylonian” iteration})$$

$$f(x) = \frac{1}{x} - a \Rightarrow x_{k+1} = x_k(2 - ax_k) \quad (\text{reciprocal, division-free})$$

(R7) False position: which endpoint moves. The new estimate x_r always replaces the endpoint whose f has the *same sign* as $f(x_r)$. If f is strongly convex/concave on the bracket, one endpoint is never that one, so it **freezes** — the bracket width stops going to zero even though x_r converges. This is why $|\epsilon_a|$, not the bracket width, must be used as the stopping test.

Quick Numbers (memorize)

Powers of 2 (for bisection counts). $2^5 = 32$, $2^{10} = 1024$, $2^{15} = 32768$, $2^{20} = 1048576$. Rule of thumb: **10 iterations $\approx$ 3 decimal digits** (since $2^{10} \approx 10^3$).

Worked iteration counts ($n \geq \log_2 \frac{b-a}{\epsilon}$, rounded up):

Bracket	[1, 2]	[0, 0.11]	[0, 1]	[2, 3]
Tolerance	10^{-3}	10^{-4}	10^{-6}	0.5×10^{-4}
n	10	11	20	15

The deck’s floating-ball benchmark ($f(x) = x^3 - 0.165x^2 + 3.993 \times 10^{-4}$, bracket $[0, 0.11]$, root 0.06238): bisection **10** iterations for 2 digits; false position **5** iterations for about 5 digits; Newton–Raphson **3** iterations with digits $0 \rightarrow 2 \rightarrow 4$. $f'(x) = 3x(x - 0.11)$ vanishes at both bracket ends.

Newton on $\sqrt{2}$ from $x_0 = 1$: 1.5, 1.416667, 1.414216, 1.4142136 — errors 8.6×10^{-2} , 2.5×10^{-3} , 2.1×10^{-6} , 1.6×10^{-12} . Each error is about 0.354 times the square of the previous one.

Exam Traps

- **Bisection’s iteration count does not depend on f** — only on $b - a$ and ϵ . Options mentioning the function are distractors.
- $f(x_\ell)f(x_u) < 0$ is **sufficient, not necessary**. $f(x) = x^2$ has a root at 0 with *no* sign change anywhere; bisection can never find it.
- A sign change does **not** guarantee a root: $f(x) = 1/x$ on $[-2, 3]$ satisfies the test but has a

singularity, not a root. Bisection converges to $x = 0$ and reports it confidently.

- “At least one root” — the IVT never promises exactly one. A bracket may contain three roots.
- Quadratic convergence means the *number of correct digits* doubles, not that the error halves.
- Newton needs f' evaluated every step; “fewer iterations” is not automatically “less work” if f' is expensive.
- False position is *not* always faster than bisection: on $\ln x$ over $[10^{-4}, 10^4]$ it took **126** iterations against bisection’s **34**.
- In false position the bracket width may never approach zero (frozen endpoint), so never use bracket width as the convergence test there.
- Newton at a multiple root is only linear — if a question says “double root,” quadratic convergence is the wrong answer.

11 Linear Systems & Eigenvalues — Beyond the Slides

11.1 Systems of Equations — Derived & Implicit Formulas

Derived & Implicit Formulas — not on the slides, but exam-critical

(S1) How many multipliers does elimination compute? Step k needs $n - k$ multipliers, so

$$\sum_{k=1}^{n-1} (n - k) = \boxed{\frac{n(n-1)}{2}}$$

($n = 3 \rightarrow 3$, $n = 4 \rightarrow 6$, $n = 5 \rightarrow 10$, $n = 10 \rightarrow 45$). This is also the number of entries stored in the strictly-lower part of L — the same objects, counted twice.

(S2) Determinant from elimination, with the sign rule. Row-addition operations do not change $\det$, and L is unit-lower-triangular so $\det L = 1$. Therefore

$$\det A = \det(LU) = \det L \cdot \det U = \prod_{i=1}^n u_{ii}$$

and with pivoting,

$$\boxed{\det A = (-1)^{\#\text{row swaps}} \prod_{i=1}^n u_{ii}}$$

An *even* number of swaps leaves the sign alone; an odd number flips it.

(S3) The deck’s clock-cycle cost model, assembled. With $T =$ one clock cycle ($+$, $-$, $\times$ cost $4T$; $\div$ costs $16T$):

$$C_T|_{DE} = \left(\frac{8}{3}n^3 + 4n^2 - \frac{20}{3}n\right) T \quad (\text{decomposition only})$$

$$C_T|_{FS} = (4n^2 - 4n)T, \quad C_T|_{BS} = (4n^2 + 12n)T$$

$$C_T|_{LU} = C_T|_{DE} + C_T|_{FS} + C_T|_{BS} = \left(\frac{8}{3}n^3 + 12n^2 + \frac{4}{3}n\right) T$$

$$C_T|_{GE} = C_T|_{FE} + C_T|_{BS} = \left(\frac{8}{3}n^3 + 12n^2 + \frac{4}{3}n\right) T = C_T|_{LU}$$

They are equal exactly, not approximately — for one right-hand side LU buys nothing.

(S4) Cost with k right-hand sides — where LU actually wins.

$$C_T|_{LU}^{(k)} = C_T|_{DE} + k(C_T|_{FS} + C_T|_{BS}), \quad C_T|_{GE}^{(k)} = k(C_T|_{FE} + C_T|_{BS})$$

LU pays the $O(n^3)$ decomposition **once**; Gauss repeats it k times. Setting $k = n$ (the matrix inverse):

$$C_T|_{inv,LU} = \left(\frac{32}{3}n^3 + 12n^2 - \frac{20}{3}n\right)T, \quad C_T|_{inv,GE} = \left(\frac{8}{3}n^4 + 12n^3 + \frac{4}{3}n^2\right)T$$

$$\frac{C_T|_{inv,GE}}{C_T|_{inv,LU}} \approx \frac{\frac{8}{3}n^4}{\frac{32}{3}n^3} = \boxed{\frac{n}{4}}$$

(S5) Why forward substitution is cheaper than back substitution. L has ones on the diagonal, so forward substitution performs **zero divisions**; back substitution performs one division per row (n in total). In a model where $\div$ costs $4\times$ a multiply, that gap is real — it is exactly the $-4n$ vs $+12n$ difference between $C_T|_{FS}$ and $C_T|_{BS}$.

(S6) Floating-point operation counts (the standard result behind the clock model).

$$\text{Gauss elimination} \sim \frac{2}{3}n^3, \quad \text{Gauss-Jordan} \sim n^3, \quad \text{each substitution} \sim n^2.$$

So Gauss-Jordan does roughly **50% more arithmetic** than Gauss elimination — the price of driving the matrix all the way to I instead of stopping at triangular.

(S7) Conditioning (named standard result; the deck shows the symptom, not the name).

$$\kappa(A) = \|A\| \|A^{-1}\| \geq 1, \quad \text{digits lost} \approx \log_{10} \kappa(A).$$

A tiny pivot inflates the multiplier, which amplifies existing round-off — the mechanism behind the deck's 0.001-pivot disaster (50% error at 5 digits, fixed exactly by one row swap).

Quick Numbers (memorize)

Inverse-cost ratio (straight off the slide, verified):

n	10	100	1000	10000
$C_T _{inv,GE}/C_T _{inv,LU}$	3.28	25.83	250.8	2501
$n/4$ (asymptote)	2.5	25	250	2500

Cost model: $+, -, \times \rightarrow 4$ cycles; $\div \rightarrow 16$ cycles. At 1.2 GHz, $T = 1/(1.2 \times 10^9) \approx 0.833$ ns.

The rocket matrix (know this one cold). $A = \begin{bmatrix} 25 & 5 & 1 \\ 64 & 8 & 1 \\ 144 & 12 & 1 \end{bmatrix}$, $\mathbf{b} = (106.8, 177.2, 279.2)$.

Multipliers $m_{21} = 2.56$, $m_{31} = 5.76$, $m_{32} = 3.5$.

$$U = \begin{bmatrix} 25 & 5 & 1 \\ 0 & -4.8 & -1.56 \\ 0 & 0 & 0.7 \end{bmatrix}, \quad L = \begin{bmatrix} 1 & 0 & 0 \\ 2.56 & 1 & 0 \\ 5.76 & 3.5 & 1 \end{bmatrix}, \quad \det A = 25(-4.8)(0.7) = -84$$

Solution $a_1 = 0.290472$, $a_2 = 19.6905$, $a_3 = 1.08571$; forward-substitution vector $Z = (106.8, -96.208, 0.76)$ — identical to Gauss's modified RHS, because $Z = L^{-1}C$.

Round-off benchmark. Pivot 0.001 gives multiplier -2750 ; after swapping, pivot -2.75 gives -0.00036 — about **seven million times smaller**. Result at 5 digits: $(0.625, 1.5, 0.99995)$ naive (50% error) vs the exact $(1, 1, 1)$ with pivoting.

Exam Traps

- **LU is not faster than Gauss for one solve.** $C_T|_{LU} = C_T|_{GE}$ *exactly*. The win appears only with multiple right-hand sides.
- **An even number of row swaps does not change det.** Count the swaps, then apply $(-1)^\#$ — do not assume pivoting always flips the sign.
- $\det L = 1$ **always**, so $\det A = \det U$. Multiplying $\det L$ by something is a wasted step and a wrong answer.
- **Partial pivoting compares $|a_{ik}|$ in the current column only**, over rows $k..n$ — not the whole remaining submatrix (that would be complete pivoting).
- **More precision does not fix a zero pivot.** Extra digits help round-off; only a swap fixes a structurally zero pivot.
- **A zero pivot can appear mid-elimination** even when the original matrix has no zero entries in the pivot column.
- **Gauss–Jordan does more work, not less.** “No back substitution” sounds cheaper and is not.
- **The multiplier count is $n(n-1)/2$, not n^2 .** For $n = 4$ that is 6, not 16.

11.2 Eigenvalues — Derived & Implicit Formulas**Derived & Implicit Formulas — not on the slides, but exam-critical**

(V1) Trace and determinant — the two fastest checks in the whole topic.

$$\boxed{\operatorname{tr} A = \sum_{i=1}^n \lambda_i}, \quad \boxed{\det A = \prod_{i=1}^n \lambda_i}$$

For a 2×2 matrix this *replaces* the characteristic polynomial entirely:

$$\lambda^2 - (\operatorname{tr} A)\lambda + \det A = 0 \implies \lambda = \frac{\operatorname{tr} A \pm \sqrt{(\operatorname{tr} A)^2 - 4 \det A}}{2}$$

“ $\operatorname{tr} = 7$, $\det = 12$ ” gives 3 and 4 in about three seconds.

(V2) The spectrum map — the single richest source of trap MCQs. If $A\mathbf{v} = \lambda\mathbf{v}$, then for the *same* eigenvector $\mathbf{v}$:

Matrix	Eigenvalue	Eigenvector
A^k	λ^k	$\mathbf{v}$ (unchanged)
A^{-1}	$1/\lambda$	$\mathbf{v}$ (unchanged)
$A + cI$	$\lambda + c$	$\mathbf{v}$ (unchanged)
cA	$c\lambda$	$\mathbf{v}$ (unchanged)
$p(A)$ (any polynomial)	$p(\lambda)$	$\mathbf{v}$ (unchanged)
$A^\top$	λ (same set)	generally <i>different</i>

So an eigenvalue of $A^3 - 2I$ is $\lambda^3 - 2$: if $\lambda = 2$, the answer is 6. **Every row here leaves the eigenvector alone except the transpose.**

(V3) Power-method convergence rate, derived. From

$$A^k \mathbf{x} = \lambda_1^k \left[c_1 \mathbf{v}_1 + \sum_{i \geq 2} c_i \left(\frac{\lambda_i}{\lambda_1} \right)^k \mathbf{v}_i \right],$$

the unwanted content decays by the factor $|\lambda_2/\lambda_1|$ *per iteration*:

$$\text{error after } k \text{ iterations} \sim \left| \frac{\lambda_2}{\lambda_1} \right|^k$$

Iterations needed to gain one decimal digit:

$$k_{\text{digit}} = \frac{1}{\log_{10} |\lambda_1/\lambda_2|}.$$

For the deck's $\begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$ ($\lambda = 3, 1$) the ratio is $1/3$: about $1/\log_{10} 3 = 2.1$ iterations per digit.

(V4) When the power method fails (both cases are exam favourites).

- **No strictly dominant eigenvalue**, e.g. $\lambda_1 = 5$, $\lambda_2 = -5$. Then $|\lambda_2/\lambda_1| = 1$, nothing decays, and the iterate oscillates forever between two directions.
- **The start vector is deficient**, $c_1 = 0$ (i.e. $\mathbf{x}_0$ is orthogonal to $\mathbf{v}_1$ in the symmetric case). In exact arithmetic the $\mathbf{v}_1$ component can never appear; the method converges to λ_2 instead.

Slow (not failed) convergence happens when $|\lambda_2|$ is merely *close* to $|\lambda_1|$.

(V5) Inverse power method, derived in one line.

$$A\mathbf{v} = \lambda\mathbf{v} \implies \mathbf{v} = \lambda A^{-1}\mathbf{v} \implies A^{-1}\mathbf{v} = \frac{1}{\lambda}\mathbf{v}$$

Reciprocation *reverses the ranking*: the smallest $|\lambda|$ of A becomes the dominant eigenvalue of A^{-1} . Run the ordinary power method on A^{-1} and invert the answer. In practice never form A^{-1} — factor $A = LU$ once and solve $A\mathbf{y}_{k+1} = \mathbf{x}_k$ each iteration (this is precisely the LU “many right-hand sides” payoff of (S4)).

(V6) Shifted power method (*extension — the deck teaches the inverse method but not the shift*).

Because $A - \sigma I$ has eigenvalues $\lambda_i - \sigma$:

- power method on $A - \sigma I$ finds the eigenvalue *farthest* from σ ;
- inverse power method on $A - \sigma I$ finds the eigenvalue *nearest* to σ , converging at rate $|\lambda_{\text{near}} - \sigma|/|\lambda_{\text{next}} - \sigma|$, which a good shift makes tiny.

(V7) Deflation. With $\hat{\mathbf{v}}_1$ normalized to unit length,

$$A_2 = A - \lambda_1 \hat{\mathbf{v}}_1 \hat{\mathbf{v}}_1^\top \quad \text{has spectrum} \quad \{0, \lambda_2, \lambda_3, \dots\}.$$

It works because $\hat{\mathbf{v}}_1^\top \hat{\mathbf{v}}_j = 0$ for $j \neq 1$ — which requires **orthogonal eigenvectors**, guaranteed for *symmetric* A . Errors compound: each stage deflates with a numerically approximate eigenvector.

(V8) Symmetric matrices (used implicitly throughout). Real eigenvalues, orthogonal eigenvectors, and for 2×2

$$A = \begin{bmatrix} a & b \\ b & d \end{bmatrix} \implies \lambda = \frac{(a+d) \pm \sqrt{(a-d)^2 + 4b^2}}{2}$$

(the discriminant is a sum of squares, so the roots are always real).

Quick Numbers (memorize)

The deck's three standing examples.

- $\begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$: $\text{tr} = 4$, $\det = 3$, $\lambda = 3, 1$; $\mathbf{v}_1 = [1, 1]^\top$, $\mathbf{v}_2 = [1, -1]^\top$; ratio $1/3$. Power method from

$[1, 0]^T$: 2.000, 2.500, 2.800, 2.929 $\rightarrow$ 3. Deflated: $A_2 = \begin{bmatrix} 0.5 & -0.5 \\ -0.5 & 0.5 \end{bmatrix}$, spectrum $\{0, 1\}$.

- 3×3 tridiagonal (3.556, -1.778): converges to $\lambda_{\max} = 6.07048$; the reported error spikes to 150% and 214% at iterations 3–4 *purely because the normalizing factor changes sign*.
- 4×4 block matrix: $\lambda = 10, 6, 4, 2$ with $\mathbf{v} = [1, 1, 0, 0], [1, -1, 0, 0], [0, 0, 1, 1], [0, 0, 1, -1]$. Inverse power method converges to $\lambda_{\min} = 2$ (normalizing factor $\rightarrow 0.5$).

Ratio-to-digits. $|\lambda_2/\lambda_1| = 0.5 \rightarrow 3.3$ iterations/digit; $= 1/3 \rightarrow 2.1$; $= 0.1 \rightarrow 1$; $= 0.9 \rightarrow 21.9$ (painfully slow).

Exam Traps

- **Shifts and powers never move the eigenvectors.** Only the transpose (and general similarity) does. Options claiming “eigenvector becomes $\mathbf{v} + c$ ” are nonsense.
- $\det(A + cI) \neq \det A + c$. You must shift each eigenvalue and re-multiply: $\prod(\lambda_i + c)$.
- **The power method finds the largest $|\lambda|$, not the largest λ .** With $\lambda = -7$ and $+5$ it converges to -7 .
- $|\lambda_1| = |\lambda_2|$ **breaks it** even though both are perfectly well-defined eigenvalues.
- **A huge reported $|\epsilon_a|$ can be a sign flip, not divergence** — the 3×3 example shows 150% and 214% mid-convergence.
- **The normalizing factor is the eigenvalue estimate**; the normalized vector is the eigenvector estimate. Do not report the vector’s largest entry (always 1) as λ .
- **Deflation needs symmetry** (orthogonal eigenvectors). Applying it blindly to a non-symmetric matrix is unjustified.
- **Inverse power method returns $1/\lambda$** — you must invert the converged normalizing factor to get $\lambda_{\min}$.
- **Never actually form A^{-1}** for the inverse power method; factor once and re-solve.

12 Optimization — Beyond the Slides

12.1 Golden Section Search — Derived & Implicit Formulas

Derived & Implicit Formulas — not on the slides, but exam-critical

(G1) Interval width after n iterations — the formula a past exam actually required. Every iteration keeps the fraction R of the current bracket, so the widths form a geometric sequence:

$$L_1 = RL_0, \quad L_2 = R^2L_0, \quad \dots \quad \boxed{L_n = R^n L_0}, \quad R = \frac{\sqrt{5} - 1}{2} = 0.618034.$$

With $L_0 = x_u - x_L$ the starting width. A question asking for the bracket after 3 iterations of $[0, 4]$ is one multiplication: $4(0.236068) = 0.944272$ — *not* three rounds of hand iteration.

(G2) Iterations required for a target width. Solve $R^n L_0 \leq \varepsilon$ (note $\ln R < 0$, so the inequality flips):

$$\boxed{n \geq \frac{\ln(\varepsilon/L_0)}{\ln R} = \frac{\log(\varepsilon/L_0)}{\log 0.618034}}$$

and round *up*. Example: $L_0 = 1$, $\varepsilon = 0.01$ gives $n \geq 9.57 \Rightarrow 10$ iterations.

(G3) Reported optimum and its uncertainty. The answer is the bracket midpoint, so the worst-case error is *half* the final width:

$$x_{opt} = \frac{x_u + x_L}{2}, \quad |x_{opt} - x^*| \leq \frac{L_n}{2} = \frac{R^n L_0}{2}.$$

(G4) Function-evaluation count. Iteration 1 needs **two** interior evaluations; every later iteration reuses one point and needs only **one**:

$$\boxed{\text{interior evaluations after } n \text{ iterations} = n + 1}$$

(Add 2 more if the problem also counts the endpoints f_L, f_u .) This is the entire point of the golden placement — state your counting convention, because MCQ options differ by exactly these ± 2 .

(G5) The three identities of R (each has appeared as an MCQ).

$$R^2 + R - 1 = 0 \implies \boxed{R^2 = 1 - R = 0.381966}, \quad \boxed{\frac{1}{R} = 1 + R = 1.618034}$$

So R^2 and $1 - R$ are the *same number*, and $1/R$ exceeds R by exactly 1.

(G6) Per-iteration reduction, and why “slower” is still better here. GSS discards $1 - R = 38.2\%$ of the bracket per iteration; bisection discards 50%. So in raw width-per-iteration, **bisection wins**. But bisection solves $f(x) = 0$ using a sign change, which an optimizer does not have; GSS needs only *comparisons* of f values, and it recycles one evaluation per round. Comparing widths:

$$\text{GSS: } 0.618^n \quad \text{vs} \quad \text{bisection-style halving: } 0.5^n$$

(G7) Absolute vs relative stopping. The deck’s criterion is *relative*:

$$\left| \frac{2(x_u - x_L)}{x_u + x_L} \right| < \varepsilon_s$$

which is L_n divided by the midpoint. On an interval far from the origin this triggers much earlier than an absolute test $L_n < \varepsilon$; near the origin it is far stricter. Read which one a question wants.

Quick Numbers (memorize)

R^n — memorize the first four, and know how to get the rest.

n	1	2	3	4	5
R^n	0.618034	0.381966	0.236068	0.145898	0.090170
0.5^n	0.5	0.25	0.125	0.0625	0.03125
n	6	7	8	9	10
R^n	0.055728	0.034442	0.021286	0.013156	0.008131
0.5^n	0.015625	0.00781	0.00391	0.00195	0.00098

Iterations to shrink a unit interval (round up):

Target ε	0.1	0.05	0.01	0.001
GSS n	5	7	10	15
Halving n	4	5	7	10

Constants: $R = 0.618034$, $R^2 = 1 - R = 0.381966$, $1/R = 1 + R = 1.618034$, $\sqrt{5} = 2.236068$.

The deck’s worked run ($f = 2 \sin x - x^2/10$ on $[0, 4]$): $d_1 = 2.47214$, $x_1 = 1.52786$, $x_2 = 2.47214$, $f_1 = 1.76580 > f_2 = 0.64586$, so $x_u \leftarrow 2.47214$. True optimum $x^* \approx 1.4276$, $f^* \approx 1.7757$.

Exam Traps

- $R = 0.618$, **not** 1.618. The interval *shrinks*; multiplying by 1.618 grows it. Both numbers will be offered.
- $L_n = R^n L_0$ — **do not forget** L_0 . The answer for $[0, 4]$ after 3 iterations is $4R^3$, not R^3 .
- **Iterations vs function evaluations are different counts** (n vs $n + 1$ interior). Read the stem.
- **The reported error is $L_n/2$, not L_n** , because the answer is the midpoint.
- **GSS removes less per iteration than bisection** (38.2% vs 50%). “Golden” refers to evaluation reuse, not to raw speed.
- **Unimodality is required**. With two maxima the comparison rule can discard the better one — the method still returns an answer, silently wrong.
- **The deck’s stopping test is relative**; comparing it directly against an absolute tolerance is a mismatch.
- $R^2 = 1 - R$, so an option offering “0.382” and one offering “ R^2 ” are the same answer.

12.2 Gradient Descent — Derived & Implicit Formulas**Derived & Implicit Formulas — not on the slides, but exam-critical**

(D1) The exact convergence condition on a quadratic — the key hidden result. For $J(x) = a(x - x^*)^2$ we have $J'(x) = 2a(x - x^*)$, so the update $x_{k+1} = x_k - \alpha J'(x_k)$ gives, in terms of the error $e_k = x_k - x^*$,

$$\boxed{e_{k+1} = (1 - 2a\alpha) e_k} \quad \implies \quad e_k = (1 - 2a\alpha)^k e_0.$$

Convergence requires $|1 - 2a\alpha| < 1$, i.e.

$$\boxed{0 < \alpha < \frac{1}{a}}$$

and the behaviour splits into four regimes:

Range of α	Factor $1 - 2a\alpha$	Behaviour	Example ($a = 3$, $x^* = 2$, $x_0 = 0$)
$0 < \alpha < \frac{1}{2a}$	in $(0, 1)$	monotone convergence	$\alpha = 0.1$: 1.2, 1.68, 1.872, ...
$\alpha = \frac{1}{2a}$	0	converges in one step	$\alpha = \frac{1}{6}$: 2.0 immediately
$\frac{1}{2a} < \alpha < \frac{1}{a}$	in $(-1, 0)$	oscillating convergence	$\alpha = 0.25$: 3.0, 1.5, 2.25, 1.875, ...
$\alpha = \frac{1}{a}$	-1	cycles forever	$\alpha = \frac{1}{3}$: 4, 0, 4, 0, ...
$\alpha > \frac{1}{a}$	< -1	diverges	$\alpha = 0.4$: 4.8, -1.92, 7.49, ...

The boundary case $\alpha = 1/a$ producing a perfect 2-cycle is prime trap material: the iterate neither converges nor blows up.

(D2) Multivariable version. With Hessian H having eigenvalues $\lambda_{\min} \leq \dots \leq \lambda_{\max}$:

$$\alpha < \frac{2}{\lambda_{\max}} \quad \text{for stability,} \quad \text{convergence factor} = \max_i |1 - \alpha \lambda_i|.$$

The slowness of ill-conditioned problems is measured by $\kappa = \lambda_{\max}/\lambda_{\min}$: large κ forces a small α (set by $\lambda_{\max}$) while progress along the flat direction is governed by $\lambda_{\min}$ — the classic zig-zag.

(D3) The analytic optimum GD is chasing (intercept-only SSR). Setting $\frac{\partial SSR}{\partial b} = -2 \sum (y_i - b - mx_i) = 0$:

$$b^* = \frac{1}{n} \sum_i (y_i - mx_i) = \overline{y - mx}.$$

For the deck's data $(1, 2), (2, 3), (3, 5)$ with $m = 1$: $b^* = \frac{1}{3}(1 + 1 + 2) = 1.3333$ — exactly what the iterates $0.8, 1.12, 1.248, \dots$ are approaching, with the step shrinking by the constant factor $1 - 2a\alpha$ each round.

(D4) Cost per step. Batch GD touches all m samples per step, $O(m)$ per parameter; strict SGD uses 1 sample, mini-batch uses b . Same update formula, different estimate of the same gradient — the noise is the price of the speed.

12.3 Newton's Method for Optimization — Derived & Implicit Formulas

Derived & Implicit Formulas — not on the slides, but exam-critical

(N1) On a quadratic, Newton lands on the optimum in exactly ONE step, from anywhere. For $f(x) = a(x - x^*)^2$: $f' = 2a(x - x^*)$, $f'' = 2a$, so

$$x_1 = x_0 - \frac{2a(x_0 - x^*)}{2a} = x^*.$$

No dependence on x_0 at all. The reason is structural: Newton minimizes the *second-order Taylor model*, and for a quadratic that model is exact. The same holds in n dimensions with $x_1 = x_0 - H^{-1}g$.

(N2) Newton solves $f' = 0$ — so it happily finds maxima and saddles. Applying it to $f(x) = -x^2 + 4x$ from $x_0 = 5$ gives $x_1 = 2$ in one step, and $f'' = -2 < 0$: that is the **maximum**. The update never inspects the sign of the curvature, which is precisely limitation (2) in the notes. Always check $f'' > 0$ (or H positive definite) before calling a stationary point a minimum.

(N3) Quadratic convergence. Newton for optimization is Newton root-finding applied to f' , so it inherits

$$e_{k+1} \approx \left| \frac{f'''(x^*)}{2f''(x^*)} \right| e_k^2,$$

i.e. the number of correct digits roughly doubles per step — the deck's $x^2 - 4\cos x$ run goes $0 \rightarrow 2 \rightarrow 5$ decimal places.

(N4) Cost comparison, the reason “when to use which” has an answer.

	Gradient Descent	Newton
Information	∇f only	∇f and H (n^2 entries)
Work per step	$O(n)$	$O(n^3)$ to solve $Hy = g$
Iterations	many (linear)	few (quadratic)
Tuning	learning rate α	none

Large n (typical ML) $\Rightarrow$ GD; small n with a cheap Hessian $\Rightarrow$ Newton.

12.4 Stationary Points & Lagrange — Derived & Implicit Formulas

Derived & Implicit Formulas — not on the slides, but exam-critical

(L1) The 2-D Hessian determinant test (*shortcut; the deck states the PSD condition, this is the practical version*). At a stationary point of $f(x, y)$, with

$$H = \begin{bmatrix} f_{xx} & f_{xy} \\ f_{xy} & f_{yy} \end{bmatrix}, \quad \det H = f_{xx}f_{yy} - f_{xy}^2 :$$

$\det H > 0$ and $f_{xx} > 0$	both eigenvalues > 0	minimum
$\det H > 0$ and $f_{xx} < 0$	both eigenvalues < 0	maximum
$\det H < 0$	eigenvalues of opposite sign	saddle
$\det H = 0$	at least one zero eigenvalue	inconclusive

Worked check on $f = x^3 + y^3 - 3xy$: stationary points $(0,0)$ and $(1,1)$. At $(0,0)$, $H = \begin{bmatrix} 0 & -3 \\ -3 & 0 \end{bmatrix}$,

$\det = -9 < 0 \Rightarrow$ saddle. At $(1,1)$, $H = \begin{bmatrix} 6 & -3 \\ -3 & 6 \end{bmatrix}$, $\det = 27 > 0$ with $f_{xx} = 6 > 0 \Rightarrow$ minimum.

(L2) λ is the shadow price — derived in the deck's own convention. Write the constraint as $C = x_1 + x_2 + c = 0$ and minimize $J = x_1^2 + x_2^2$. Solving the Lagrange system in general:

$$x_1 = x_2 = -\frac{c}{2}, \quad \lambda = c, \quad J^* = \frac{c^2}{2}$$

$$\Rightarrow \boxed{\frac{dJ^*}{dc} = c = \lambda}$$

So λ measures exactly how much the optimal objective moves per unit change in the constraint constant. At $c = 3$ (the deck's example) $\lambda = 3$ and $dJ^*/dc = 3$: tightening the constraint by 0.1 raises the optimal cost by about 0.3.

(L3) Bordered Hessian sign rule (2 variables, 1 constraint).

$$\bar{H} = \begin{bmatrix} 0 & C_{x_1} & C_{x_2} \\ C_{x_1} & \Lambda_{x_1 x_1} & \Lambda_{x_1 x_2} \\ C_{x_2} & \Lambda_{x_2 x_1} & \Lambda_{x_2 x_2} \end{bmatrix} \quad \boxed{\det \bar{H} < 0 \Rightarrow \text{minimum}, \quad \det \bar{H} > 0 \Rightarrow \text{maximum}}$$

Verified on the deck's example: $J = x_1^2 + x_2^2$, $C = x_1 + x_2 + 3$ gives $\bar{H} = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 2 & 0 \\ 1 & 0 & 2 \end{bmatrix}$ with

$\det = -4 < 0$ — a minimum, matching the known answer $(-1.5, -1.5)$. Flipping to $J = -(x_1^2 + x_2^2)$ gives $\det = +4 > 0$, a maximum.

(L4) Housekeeping that costs marks. A constraint given as $C(\mathbf{x}) = g$ must first be rewritten $C(\mathbf{x}) - g = 0$. The number of equations is always (number of variables) + (number of constraints), because $\partial \Lambda / \partial \lambda = 0$ simply returns the constraint.

Quick Numbers (memorize)

GD regimes at a glance (for $J = a(x - x^*)^2$, factor $q = 1 - 2a\alpha$):

$$\alpha = \frac{1}{2a} \ (q = 0) \text{ one step} \quad | \quad \alpha = \frac{1}{a} \ (q = -1) \text{ 2-cycle} \quad | \quad \alpha > \frac{1}{a} \text{ diverges}$$

Newton facts: exactly 1 step on any quadratic; digits double otherwise; finds *any* stationary point; $O(n^3)$ per step.

Lagrange: $\lambda = dJ^*/dc$; $\det \bar{H} < 0 \Rightarrow \min$, $> 0 \Rightarrow \max$; the deck's example gives $x_1 = x_2 = -1.5$, $\lambda = 3$, $\det \bar{H} = -4$.

Hessian test: $\det H < 0$ always means saddle, regardless of f_{xx} .

Exam Traps

- **A larger learning rate is not “faster”.** Past $\alpha = 1/(2a)$ the iterates oscillate; at $\alpha = 1/a$ they cycle forever; beyond that they diverge.
- $\alpha = 1/a$ is not “marginally convergent” — **it never converges.** It produces an exact 2-cycle $(4, 0, 4, 0, \dots)$.
- **Newton converging in one step does not mean it found a minimum.** On $-x^2 + 4x$ it reaches the *maximum* in one step.
- **Newton needs no learning rate**, so “tune α ” options are wrong for it; conversely GD needs no Hessian.
- **Zero gradient is necessary, never sufficient.** x^3 at 0 has $J' = 0$ and is a saddle.
- $\det H = 0$ is **inconclusive**, not “saddle”.
- **The parallel-gradient condition only *locates* a constrained stationary point**; classification needs the bordered Hessian.
- **Sign of λ follows your sign convention.** State whether the constraint was written $C = 0$ or $C = g$ before quoting λ as a shadow price.
- **In the bordered Hessian the $(1, 1)$ entry is 0**, because $\partial^2 \Lambda / \partial \lambda^2 = 0$ — the constraint has no λ in it.

13 Regression & Interpolation — Beyond the Slides**13.1 Regression — Derived & Implicit Formulas****Derived & Implicit Formulas — not on the slides, but exam-critical**

Notation fixed once (read this first). For data (x_i, y_i) , $i = 1..n$, fit $\hat{y} = mx + b$. Write

$$\begin{aligned}\bar{x} &= \frac{1}{n} \sum x_i, & \bar{y} &= \frac{1}{n} \sum y_i, \\ S_{xx} &= \sum (x_i - \bar{x})^2, & S_{yy} &= \sum (y_i - \bar{y})^2, \\ S_{xy} &= \sum (x_i - \bar{x})(y_i - \bar{y}), & e_i &= y_i - \hat{y}_i.\end{aligned}$$

Three sums of squares — **name them explicitly in the exam, books disagree:**

$$\begin{aligned}\text{SST} &= \sum (y_i - \bar{y})^2 = S_{yy} && \text{(total)} \\ \text{SSE} &= \sum (y_i - \hat{y}_i)^2 = \sum e_i^2 && \text{(residual / error)} \\ \text{SSR} &= \sum (\hat{y}_i - \bar{y})^2 && \text{(regression / explained)}\end{aligned}$$

The deck writes SSE for the quantity least squares *minimises*; that is the residual one above, and this sheet keeps that meaning everywhere. If a question writes “SSR” to mean “sum of squared residuals”, it means our SSE — read the words, not the letters.

(R1) The fitted line ALWAYS passes through the centroid $(\bar{x}, \bar{y})$. Straight from the intercept normal equation $b = \frac{\sum y - m \sum x}{n} = \bar{y} - m\bar{x}$:

$$\hat{y}(\bar{x}) = m\bar{x} + b = m\bar{x} + (\bar{y} - m\bar{x}) = \bar{y}.$$

So $b = \bar{y} - m\bar{x}$ is a second, much faster route to the intercept, and $(\bar{x}, \bar{y})$ is the one point you can name without computing anything. *Instant MCQ solver.*

(R2) Residuals of a least-squares fit *with* an intercept satisfy

$$\sum_{i=1}^n e_i = 0 \quad \text{and} \quad \sum_{i=1}^n x_i e_i = 0.$$

These two *are* the normal equations: $\partial \text{SSE} / \partial b = -2 \sum e_i = 0$ and $\partial \text{SSE} / \partial m = -2 \sum x_i e_i = 0$. Consequences: $\bar{e} = 0$, $\sum \hat{y}_i = \sum y_i$, and $\sum \hat{y}_i e_i = 0$ (residuals are “orthogonal” to the fit). (R1) is just the first identity divided by n . *If a question hands you residuals that do not sum to zero, that line is not the least-squares line.*

(R3) Slope $\leftrightarrow$ correlation link (get r without the big formula). With sample standard deviations $s_x = \sqrt{S_{xx}/(n-1)}$ and $s_y = \sqrt{S_{yy}/(n-1)}$:

$$m = \frac{S_{xy}}{S_{xx}} = r \frac{s_y}{s_x} \quad \Longleftrightarrow \quad r = m \frac{s_x}{s_y} = \frac{S_{xy}}{\sqrt{S_{xx}S_{yy}}}.$$

Since $s_x, s_y > 0$, $\text{sign}(r) = \text{sign}(m)$ always. The $(n-1)$ factors cancel, so $r = S_{xy}/\sqrt{S_{xx}S_{yy}}$ works with raw centred sums.

(R4) Variance decomposition and what r^2 really is.

$$\text{SST} = \text{SSR} + \text{SSE}$$

$$r^2 = \frac{\text{SSR}}{\text{SST}} = 1 - \frac{\text{SSE}}{\text{SST}} = \text{fraction of variance in } y \text{ explained.}$$

Hence $0 \leq r^2 \leq 1$ always, and $\text{SSE} = (1 - r^2) \text{SST}$ — a one-line route to SSE once you have r .

(R5) Shortcut values of SSR and SSE (no residual table needed).

$$\text{SSR} = m^2 S_{xx} = m S_{xy} = \frac{S_{xy}^2}{S_{xx}},$$

$$\text{SSE} = S_{yy} - m S_{xy} = S_{yy} - \frac{S_{xy}^2}{S_{xx}}.$$

Usually the fastest hand route in an exam: build S_{xx}, S_{yy}, S_{xy} once and everything ($m, r, r^2, \text{SSR}, \text{SSE}, s_{y/x}$) falls out.

(R6) Standard error of the estimate (spread of points about the line).

$$s_{y/x} = \sqrt{\frac{\text{SSE}}{n-2}} = \sqrt{\frac{\sum (y_i - \hat{y}_i)^2}{n-2}}.$$

Why $n-2$? The residuals obey the two exact constraints of (R2), so only $n-2$ of them are free; two degrees of freedom were spent estimating m and b . Note that $n=2$ gives $0/0$: two points are fitted perfectly and carry no information about scatter.

(R7) What survives a change of units (rescaling law). Replace $x \rightarrow px + q$ and $y \rightarrow uy + v$ with $p, u > 0$. Then

$$m' = \frac{u}{p} m, \quad b' = u b + v - \frac{u q}{p} m,$$

$$\boxed{r' = r}, \quad \boxed{r'^2 = r^2}.$$

So r and r^2 are *unitless and invariant* under any positive linear rescaling of either axis, while m and b are *not*. (If exactly one of p, u is negative, r flips sign; if both flip, r is unchanged.) Also $s_{y/x}$ scales by u — it carries the units of y .

(R8) Degenerate cases worth memorising.

- All points exactly collinear (non-horizontal) $\iff r = \pm 1 \iff \text{SSE} = 0 \iff r^2 = 1 \iff s_{y/x} = 0$.
- Best fit is a horizontal line ($m = 0$) $\iff S_{xy} = 0 \iff r = 0 \iff r^2 = 0$; then $\text{SSR} = 0$, $\text{SSE} = \text{SST}$ and $b = \bar{y}$.
- $S_{xx} = 0$ (all x_i equal): m and r are *undefined* — a vertical cloud has no least-squares line of the form $y = mx + b$.

(R9) Multiple linear regression — the $\frac{1}{2m}$ is cosmetic. For any constant $c > 0$,

$$\arg \min_{\mathbf{w}} c \sum_i (\hat{y}_i - y_i)^2 = \arg \min_{\mathbf{w}} \sum_i (\hat{y}_i - y_i)^2.$$

So $J = \frac{1}{2m} \sum (\hat{y}_i - y_i)^2$ has *exactly the same minimiser* as the plain SSE. What $\frac{1}{2m}$ changes is the *size* of the gradient, hence the useful range of α : scaling J by c and α by $1/c$ gives an identical GD trajectory.

(R10) MLR gradient and update, all faces (with $x_{i0} = 1$):

$$\frac{\partial J}{\partial w_j} = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i) x_{ij},$$

$$w_j \leftarrow w_j - \frac{\alpha}{m} \sum_{i=1}^m (\hat{y}_i - y_i) x_{ij}.$$

Matrix form: $\nabla J = \frac{1}{m} X^\top (X\mathbf{w} - \mathbf{y})$; the bias w_0 uses $x_{i0} = 1$, so it moves by the plain mean residual. *Extension (beyond the deck, but it explains assumption 5)*: setting $\nabla J = 0$ gives the normal equation $X^\top X \mathbf{w} = X^\top \mathbf{y}$. Perfect multicollinearity makes $X^\top X$ singular, so $\mathbf{w}$ is not unique — that is *why* “no multicollinearity” is an assumption.

(R11) Assumption $\rightarrow$ symptom lookup table (scenario questions).

Scenario in the question	Assumption broken	One-line justification
Errors are Poisson / skewed / counts	Normality	errors not $\sim \mathcal{N}$
Residual spread fans out with x	Homoscedasticity	error variance not constant
A few points sit extremely far off	Homoscedasticity	non-constant spread
Two inputs are nearly proportional	No multicollinearity	columns linearly dependent
Today's error carries into tomorrow	Independence	samples not independent
Repeated measures on one subject	Independence	correlated observations
True law is $y = w_1 x^2$, model is linear	Linearity	wrong functional form

Memory hook: **L-I-H-N-M** — Linearity, Independence, Homoscedasticity, Normality, Multicollinearity.

Quick Numbers (memorize)

Two fully worked reference fits — know these cold.

(A) Deck dataset $(1, 2), (2, 4), (3, 5), (4, 4), (5, 5)$, $n = 5$.

$$\sum x = 15, \sum y = 20, \sum x^2 = 55, \sum y^2 = 86, \sum xy = 66$$

$$m = 0.6, \quad b = 2.2, \quad r = 0.7746, \quad r^2 = 0.60$$

$$(\bar{x}, \bar{y}) = (3, 4) \text{ lies on the line; } S_{xx} = 10, S_{yy} = 6, S_{xy} = 6$$

$$\text{SST} = 6, \quad \text{SSR} = 3.6, \quad \text{SSE} = 2.4,$$

$$s_{y/x} = \sqrt{2.4/3} = 0.8944$$

Residuals $(-0.8, 0.6, 1.0, -0.6, -0.2)$: sum = 0 and $\sum x_i e_i = 0$. ✓

(B) Clean-number dataset $(1, 2), (2, 4), (3, 6), (4, 8), (5, 15)$, $n = 5$.

$$\sum x = 15, \sum y = 35, \sum x^2 = 55, \sum y^2 = 345, \sum xy = 135$$

$$m = 3, \quad b = -2, \quad \hat{y} = 3x - 2, \quad (\bar{x}, \bar{y}) = (3, 7)$$

$$S_{xx} = 10, \quad S_{yy} = 100, \quad S_{xy} = 30$$

$$\text{SST} = 100, \quad \text{SSR} = 90, \quad \text{SSE} = 10,$$

$$r^2 = 0.9, \quad r = 0.9487$$

$$s_x = 1.5811, \quad s_y = 5, \quad r = m \frac{s_x}{s_y} = 3 \cdot \frac{1.5811}{5} = 0.9487 \quad \checkmark$$

$$s_{y/x} = \sqrt{10/3} = 1.8257$$

Residuals $(1, 0, -1, -2, 2)$: sum = 0 and $\sum x_i e_i = 1 + 0 - 3 - 8 + 10 = 0$. ✓

Rescaling check on (A): $x \rightarrow 10x$ gives $m = 0.06$; $y \rightarrow 10y$ gives $m = 6$; $x \rightarrow 2x + 1$ gives $m = 0.3$. In all three cases $r = 0.7746$, unchanged.

Reference roots (for going $r^2 \rightarrow r$): $\sqrt{0.5} = 0.7071$, $\sqrt{0.6} = 0.7746$, $\sqrt{0.64} = 0.8$, $\sqrt{2/3} = 0.8165$, $\sqrt{0.8} = 0.8944$, $\sqrt{0.9} = 0.9487$.

Exam Traps

- Trap.** “ $r = 0.8$ means the model explains 80% of the variance.” No — $r^2 = 0.64$ does, i.e. 64%. Always square first.
- Trap.** “ $r = 0$ means x and y are unrelated.” r measures only the *linear* relation. Points on a perfect parabola symmetric about $\bar{x}$ have $r = 0$ yet are perfectly related.
- Trap.** “A high r proves x causes y .” Correlation is not causation — coincidence, or a hidden third factor driving both.
- Trap.** “Change the units of x (m to cm) and r changes.” r and r^2 are invariant; only m , b and $s_{y/x}$ move (R7).
- Trap.** “Residuals always sum to zero.” Only for a least-squares fit that *contains an intercept*. Force the line through the origin ($\hat{y} = mx$) and $\sum e_i$ is generally nonzero: on dataset (B), $m = \sum xy / \sum x^2 = 27/11$ and $\sum e_i = -20/11 \neq 0$ (though $\sum x_i e_i = 0$ still holds).
- Trap.** “ $s_{y/x} = \sqrt{\text{SSE}/n}$.” It is $\sqrt{\text{SSE}/(n-2)}$ — two degrees of freedom are consumed by m and b .
- Trap.** “SSE sits in the numerator of r^2 .” $r^2 = \text{SSR}/\text{SST} = 1 - \text{SSE}/\text{SST}$. Mixing the two inverts the answer: 0.9 becomes 0.1.
- Trap.** “Dropping the $\frac{1}{2m}$ from J changes the best weights.” No — a positive constant cannot move an arg min (R9). It only rescales the gradient, so you need a different α .
- Trap.** “MLR requires y to be normally distributed.” The Normality assumption is about the *errors*, not about y and not about the features.
- Trap.** “Adding a feature that is a copy of another is harmless.” That is exact multicollinearity: $X^\top X$ becomes singular and the individual weights are not identifiable, even though the fitted values may look fine.

11. **Trap.** “Least squares minimises the perpendicular distance to the line.” It minimises *vertical* distance, $\sum (y_i - \hat{y}_i)^2$, only.
12. **Trap.** “ $m = r s_y / s_x$ can be flipped to regress x on y .” Regressing x on y gives a *different* line (slope $r s_x / s_y$); the two lines coincide only when $r = \pm 1$. They always cross at the centroid.

13.2 Interpolation — Derived & Implicit Formulas

Derived & Implicit Formulas — not on the slides, but exam-critical

Throughout: k data points $(x_0, y_0), \dots, (x_n, y_n)$ with $k = n + 1$, so the interpolating polynomial has degree *at most* n .

(I1) Uniqueness — the single most useful fact. Through $k = n + 1$ distinct nodes there is **exactly one** polynomial of degree $\leq n$. Therefore

$$P_n^{\text{Lagrange}}(x) \equiv P_n^{\text{Newton}}(x) \quad \text{for every } x.$$

They are the *same polynomial in two different spellings*. Expand either and the coefficients agree. So an MCQ asking “do Lagrange and Newton give different values at $x = 2.5$?” answers itself: **no**. (Proof sketch: if P and Q both interpolate, $P - Q$ has degree $\leq n$ but $n + 1$ roots, so $P - Q \equiv 0$.)

(I2) Lagrange basis identities (pure MCQ fodder).

$$L_j(x) = \prod_{\substack{k=0 \\ k \neq j}}^n \frac{x - x_k}{x_j - x_k}, \quad P_n(x) = \sum_{j=0}^n y_j L_j(x).$$

- Number of basis polynomials = $k = n + 1$ — one per *point*, not per interval.
- Each L_j has degree exactly n : n factors on top, a constant $\prod_{k \neq j} (x_j - x_k)$ underneath.
- $L_j(x_i) = \delta_{ij}$: equal to 1 at its own node, 0 at every other node.
- **Partition of unity:** $\sum_{j=0}^n L_j(x) = 1$ for *all* x . (Why: interpolating the constant function $y \equiv 1$ must return 1, and uniqueness forces it.) Corollary: if every y_j equals c , then $P_n \equiv c$.
- Same argument with $f(x) = x$: $\sum_j x_j L_j(x) = x$. Handy arithmetic check.

(I3) Divided differences $\leftrightarrow$ derivatives (*extension beyond the deck, but it turns whole MCQs into one-liners*). For smooth f there is a ξ in the spanned interval with

$$f[x_0, x_1, \dots, x_n] = \frac{f^{(n)}(\xi)}{n!}.$$

Consequences to use directly:

- If f is a polynomial of degree exactly n with leading coefficient a_n , then $f^{(n)} \equiv n! a_n$, so

$$f[x_0, \dots, x_n] = a_n \quad (\text{CONSTANT, whatever the nodes}).$$

The whole n -th divided-difference column is that same number.

- The $(n+1)$ -th divided difference of a degree- n polynomial is **exactly** 0, since $f^{(n+1)} \equiv 0$. The next column is a row of zeros and the Newton coefficient $a_{n+1} = 0$: one more sample point adds *nothing*.
- Read backwards: the **leading coefficient of the Newton interpolating polynomial equals the top divided difference**, $a_n = f[x_0, \dots, x_n]$.

(I4) Equally spaced nodes: divided $\rightarrow$ forward differences. If $x_i = x_0 + ih$ with constant spacing h , then

$$f[x_0, x_1, \dots, x_n] = \frac{\Delta^n f_0}{n! h^n},$$

where $\Delta f_i = f_{i+1} - f_i$ and Δ^n is the n -th forward difference. Special case $n = 1$: $f[x_0, x_1] = \Delta f_0 / h$, the plain slope. Combined with (I3), for a degree- n polynomial $\Delta^n f_0 = n! h^n a_n$ is constant and $\Delta^{n+1} f_0 = 0$. Note the divided difference does *not* depend on h , but $\Delta^n f_0$ does.

(I5) Interpolation error term (*extension: not on the slides, but it is the formal statement behind “the fit is exact” and behind Runge*). For $f \in C^{n+1}$ and any x there is a ξ in the interval spanned by $x, x_0, \dots, x_n$ with

$$f(x) - P_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{i=0}^n (x - x_i).$$

- If f is itself a polynomial of degree $\leq n$, then $f^{(n+1)} \equiv 0$ and the error is **exactly zero everywhere**: $P_n \equiv f$. That is why 3 points from $y = x^2$ reproduce x^2 and 4 points from $y = x^3$ reproduce x^3 .
- The product $\prod (x - x_i)$ vanishes at every node — the interpolant is exact *at* the data, as it must be.
- The product grows fast *outside* the node range, so **extrapolation** is far worse than interpolation.

(I6) Cost and the recalculation-free property.

	Build P_n from scratch	Add one new point
Lagrange	$O(n^2)$	rebuild <i>every</i> L_j : $O(n^2)$ again
Newton	$O(n^2)$ (the table)	one new bottom row + one term: $O(n)$

The old Newton coefficients $a_0, \dots, a_n$ are *untouched* when (x_{n+1}, y_{n+1}) arrives; only a_{n+1} is new. Evaluating either form at one x is $O(n)$ (Horner-style nesting for Newton).

(I7) Spline counting, fully general. For k data points,

$$n = k - 1 \text{ intervals} = \text{number of spline pieces}$$

and there are $k - 2$ *interior* knots. Per piece: linear 2 unknowns, quadratic 3, cubic 4.

$$\text{Linear: } 2n = \underbrace{2n}_{\text{through pts}} \quad (\text{no extra conditions needed})$$

$$\text{Quadratic: } 3n = \underbrace{2n}_{\text{through pts}} + \underbrace{(n-1)}_{P' \text{ match}} + \underbrace{1}_{P''=0 \text{ at one end}}$$

$$\text{Cubic: } 4n = \underbrace{2n}_{\text{through pts}} + \underbrace{(n-1)}_{P' \text{ match}} + \underbrace{(n-1)}_{P'' \text{ match}} + \underbrace{2}_{\text{boundary}}$$

“Through points” is $2n$, *not* k : each piece is pinned at both of its own ends, so every interior point is used twice — once by the piece on its left, once by the piece on its right. Derivative conditions are counted at the $n - 1$ interior knots.

(I8) Smoothness ladder — what each order actually guarantees.

Spline	Continuous	P' matches at knots	P'' matches at knots
Linear	yes	no (kinks / corners)	no
Quadratic	yes	yes	no (curvature jumps)
Cubic	yes	yes	yes

A linear spline is continuous but *not differentiable* at the knots. Higher order means smoother, but more unknowns, more computation and more chance of error.

(I9) Cubic-spline boundary choices — the count is $4n$ either way.

$$\text{Natural: } P_1''(x_{\text{first}}) = 0 \quad \text{AND} \quad P_n''(x_{\text{last}}) = 0 \quad (2 \text{ equations})$$

$$\text{Clamped: } P_1'(x_{\text{first}}) = f'_{\text{start}} \quad \text{AND} \quad P_n'(x_{\text{last}}) = f'_{\text{end}} \quad (2 \text{ equations})$$

Natural means zero curvature (straight) at both ends; clamped needs *extra data*, the two end slopes, which must be given to you. For a *quadratic* spline only **one** extra condition fits ($3n$ versus $2n + (n - 1)$), which is why the deck sets $P'' = 0$ at *one* end only — a quadratic spline is inherently lopsided.

(I10) Linear interpolation is the $n = 1$ case of everything.

$$y_m = y_2 + \frac{y_3 - y_2}{x_3 - x_2}(x_m - x_2) = y_2 \frac{x_m - x_3}{x_2 - x_3} + y_3 \frac{x_m - x_2}{x_3 - x_2},$$

i.e. Lagrange with two nodes = Newton with $a_0 = y_2$, $a_1 = f[x_2, x_3]$ = the linear-spline piece on $[x_2, x_3]$. All four names, one formula. Writing $t = \frac{x_m - x_2}{x_3 - x_2} \in [0, 1]$ gives the weighted-average face

$$y_m = (1 - t)y_2 + ty_3,$$

so y_m always lies *between* y_2 and y_3 when you interpolate (and outside them when you extrapolate).

(I11) Runge's phenomenon, stated precisely enough to beat the traps. For *equally spaced* nodes on a fixed interval, raising the degree n can make the global interpolant *worse*: the oscillation is largest near the **edges** of the interval, while the middle keeps improving. Classic example $f(x) = \frac{1}{1+x^2}$ on $[-5, 5]$, where the fit wiggles and even goes negative although $f > 0$ everywhere. More points does *not* mean more accuracy for one global polynomial. The course fix: drop the single global polynomial and use *local*, low-order pieces — splines.

Quick Numbers (memorize)

Spline unknowns / equations for k data points ($n = k - 1$ pieces):

k pts	n pieces	Linear ($2n$)	Quadratic ($3n$)	Cubic ($4n$)
3	2	4	$6 = 4 + 1 + 1$	$8 = 4 + 1 + 1 + 2$
4	3	6	$9 = 6 + 2 + 1$	$12 = 6 + 2 + 2 + 2$
5	4	8	$12 = 8 + 3 + 1$	$16 = 8 + 3 + 3 + 2$
10	9	18	$27 = 18 + 8 + 1$	$36 = 18 + 8 + 8 + 2$
11	10	20	$30 = 20 + 9 + 1$	$40 = 20 + 9 + 9 + 2$
k	$k - 1$	$2(k - 1)$	$3(k - 1)$	$4(k - 1)$

In terms of k the cubic split reads $4(k - 1) = 2(k - 1) + (k - 2) + (k - 2) + 2$.

Divided-difference tables to know by heart.

$y = x^3$ at $x = 1, 2, 3, 4, 5$ (so $h = 1$ and the leading coefficient is $a_3 = 1$):

x	y	1st	2nd	3rd	4th
1	1	7	6	1	0
2	8	19	9	1	
3	27	37	12		
4	64	61			
5	125				

The 3rd column is constant = 1 = leading coefficient; the 4th column is exactly 0. Newton form from the top diagonal:

$$P_3(x) = 1 + 7(x-1) + 6(x-1)(x-2) + 1(x-1)(x-2)(x-3),$$

and $P_3(2.5) = 15.625 = 2.5^3$. ✓

Forward-difference cross-check (I4) on the same data, $h = 1$, $n = 3$: $\Delta^3 f_0 = 6$ and $n! h^n = 6$, so $f[x_0..x_3] = 6/6 = 1$. ✓ With $h = 2$ ($y = x^3$ at $x = 0, 2, 4, 6$): $\Delta^3 f_0 = 48$ and $n! h^n = 6 \cdot 8 = 48$, ratio = 1 again.

$f(x) = 2x^2 - 3x + 1$ at $x = 0, 1, 2, 3$ gives $y = 1, 0, 3, 10$; the 2nd column is constant 2 = leading coefficient and the 3rd column is 0. Top diagonal (1, -1, 2, 0).

Lagrange = Newton, one worked check. Points (0, 1), (1, 3), (2, 2), (4, 5). Divided differences $a_0 = 1$, $a_1 = 2$, $a_2 = -\frac{3}{2}$, $a_3 = \frac{7}{12}$:

$$\begin{aligned} P_3(x) &= 1 + 2x - \frac{3}{2}x(x-1) + \frac{7}{12}x(x-1)(x-2) \\ &= \frac{7}{12}x^3 - \frac{13}{4}x^2 + \frac{14}{3}x + 1. \end{aligned}$$

Both forms give $P_3(3) = \frac{3}{2}$ and $P_3(2.5) = \frac{47}{32} = 1.46875$. Lagrange weights at $x = 3$: $L_0 = \frac{1}{4}$, $L_1 = -1$, $L_2 = \frac{3}{2}$, $L_3 = \frac{1}{4}$ — they sum to 1 ✓, and $\frac{1}{4}(1) - 1(3) + \frac{3}{2}(2) + \frac{1}{4}(5) = \frac{3}{2}$ ✓. Note $L_1 < 0$: basis weights need *not* be positive.

Error term, an exact demo. $f(x) = x^4$ at nodes 0, 1, 2, 3 gives $P_3(x) = 6x^3 - 11x^2 + 6x$. Here $f^{(4)}/4! = 24/24 = 1$, so the error is exactly $\prod (x - x_i)$:

$$x = 1.5 : f = 5.0625, P_3 = 4.5, \text{err} = 0.5625 = (1.5)(0.5)(-0.5)(-1.5) \quad \checkmark$$

$$x = 4 \text{ (extrapolation)} : f = 256, P_3 = 232, \text{err} = 24 = (4)(3)(2)(1) \quad \checkmark$$

Runge numbers for $f(x) = \frac{1}{1+x^2}$ on $[-5, 5]$, equally spaced nodes, degree n (all computed):

n	$\max f - P_n $	$P_n(4.8)$ vs $f = 0.0416$	$P_n(0.1)$ vs $f = 0.99010$
4	0.44	-0.126	0.99829
8	1.05	-0.794	0.99473
10	1.92	+1.804	0.99328
16	14.4	-14.010	0.99095
20	59.8	-50.864	0.99042

Read it twice: the *edge* error explodes as n grows while the *centre* steadily improves. That contrast is Runge's phenomenon.

Exam Traps

1. **Trap.** “Lagrange and Newton give different answers.” They produce the *identical* polynomial, hence identical values at every x (I1). Only the arithmetic route and the update cost differ.
2. **Trap.** “ k points force a degree- $(k - 1)$ polynomial.” *At most* $k - 1$. Collinear points collapse to a line; 4 points sampled from $y = x^2$ give back x^2 (degree 2) and the 3rd divided difference is 0.
3. **Trap.** “ k data points give k splines.” k points give $n = k - 1$ pieces. This single off-by-one wrecks every counting MCQ: 10 points means 9 cubic pieces and 36 unknowns, not 40.
4. **Trap.** “A cubic spline through k points needs $4k$ equations.” It needs $4n = 4(k - 1)$.
5. **Trap.** “The ‘through points’ count is k .” It is $2n$: each piece is pinned at both of its own endpoints, so interior points are used twice.
6. **Trap.** “Natural and clamped cubic splines need different numbers of equations.” Both supply exactly 2 boundary equations and the total is $4n$ either way. Only *which* two conditions differ.
7. **Trap.** “A linear spline is smooth because it is continuous.” Continuous is not smooth. Linear splines have corners: P' jumps at every interior knot.
8. **Trap.** “Quadratic splines match second derivatives too.” Only cubic splines do. Quadratic matches values and first derivatives, then needs one $P'' = 0$ end condition to close the count.
9. **Trap.** “More interpolation points always means more accuracy.” False for one *global* polynomial on equally spaced nodes — Runge. The error grows at the *edges* as n rises, even while the centre improves.
10. **Trap.** “Runge’s oscillation is worst in the middle.” It is worst at the *edges* of the equally spaced grid.
11. **Trap.** “One zero entry in the 2nd divided-difference column means the data is linear.” You need the *whole* column to vanish. Example: $f = 2x^3 - x + 5$ at $x = -1, 0, 1, 2$ has $f[x_0, x_1, x_2] = 0$ but $f[x_1, x_2, x_3] = 6$, and the top divided difference is 2 — a genuine cubic.
12. **Trap.** “The last Newton coefficient is y_n .” It is the *top* divided difference $f[x_0, \dots, x_n]$, which also equals the leading coefficient of the polynomial. Only $a_0 = y_0$.
13. **Trap.** “Adding a point to a Lagrange fit is cheap, like Newton.” Every L_j gains a factor and every denominator changes: a full $O(n^2)$ rebuild. Newton appends one row and one term.
14. **Trap.** “Interpolation is just regression with more points.” Interpolation passes through *every* point exactly (all residuals zero); regression deliberately does not, so it can absorb noise. Forcing a high-degree interpolant onto noisy data over-fits the noise.
15. **Trap.** “ $\sum_j L_j(x) = 1$, so each $L_j(x)$ lies in $[0, 1]$.” They sum to 1, but individual weights can be negative or exceed 1 — e.g. $L_1(3) = -1$ and $L_2(3) = 1.5$ in the worked check above.
16. **Trap.** “Splines beat global polynomials because they are higher order.” The opposite: splines win by being *lower* order and *local*. Order is capped at 3 no matter how many points arrive.

14 Simulation, RNG & Monte Carlo — Beyond the Slides

14.1 Simulation & Modeling — Derived & Implicit Formulas

Derived & Implicit Formulas — not on the slides, but exam-critical

(D1) Little’s Law — *named standard queueing result; used here as an extension, the deck never states it.* For any queueing system observed long enough,

$$L = \lambda W, \quad L_q = \lambda W_q.$$

L = time-average number in *system*, L_q = time-average number in *queue*, λ = arrival rate, W = average time in system, W_q = average delay in queue. In the deck’s own notation $L_q \equiv \hat{q}(n)$ and $W_q \equiv \hat{d}(n)$, so

$$\hat{q}(n) \approx \lambda \hat{d}(n), \quad \lambda \approx \frac{\# \text{ arrivals in } [0, T]}{T}.$$

Companion identities (both follow immediately):

$$W = W_q + E[S], \quad L = L_q + \rho.$$

(D2) The exact finite-run version of Little’s Law. Little’s Law is a *limit* statement, but a finite run obeys an exact bookkeeping identity:

$$\int_0^T Q(t) dt = \sum (\text{time each customer spent in queue during } [0, T]).$$

So $\lambda \hat{d}(n)$ reproduces $\hat{q}(n)$ *exactly* only when the customer set in the numerator matches the one generating the area. Two safe ways to make it exact:

- Count only the n customers whose delays completed, and use only *their* share of the area.
- Count *all* arrivals in $[0, T]$ and use each one’s queue time so far, including those still waiting at T .

A short run mismatches because customers still standing in the queue at time T have contributed area but no finished delay.

(D3) Rates from mean times (implicit everywhere in the deck).

$$\lambda = \frac{1}{E[A]} \quad (\text{arrival rate}), \quad \mu = \frac{1}{E[S]} \quad (\text{service rate}).$$

$$\rho = \frac{\lambda}{\mu} = \frac{E[S]}{E[A]} = \text{offered load} \approx \hat{u}(n)$$

(the last approximation holds in steady state for a single server). **Stability condition:** $\lambda < \mu$, i.e. $\rho < 1$. If $\lambda > \mu$, work arrives faster than it can be cleared, the backlog grows by about $(\lambda - \mu)$ customers per unit time, so $Q(t) \rightarrow \infty$ — the deck’s “explosive queue.” If $\rho \geq 1$ no steady state exists and $\hat{u} \rightarrow 1$.

(D4) Time-average (continuous-time) vs. simple average (discrete-time).

$$\text{time average of } X(t) \text{ on } [0, T] = \frac{1}{T} \int_0^T X(t) dt = \frac{\sum_j x_j \Delta t_j}{\sum_j \Delta t_j},$$

i.e. a *duration-weighted* average. A plain arithmetic mean of the values seen at event times is the special case $\Delta t_j = \text{constant}$, which is almost never true — that is exactly where the bias comes from.

(D5) Area-accumulation rule and the strict update order.

$$\text{new area} = (\text{old value of } Q(t) \text{ or } B(t)) \times (\text{clock} - \text{time of last event}).$$

Order at every event: (1) areas, using old state $\rightarrow$ (2) state variables $\rightarrow$ (3) time-of-last-event $\rightarrow$ (4) event list. Since $B(t) \in \{0, 1\}$, $\int_0^T B dt$ is just the total busy time, so $\hat{u}(n)$ never needs rectangles — add up the busy intervals.

(D6) Event counting in a finite DES run. With a arrivals and d departures processed,

$$\text{total events} = a + d$$

(initialization e_0 is normally not counted). Always $d \leq a$, and $a - d$ is the number still in the system (in service or queued) at time T . Stopping after n completed *delays* is not the same as n completed *services*: a customer's delay ends when service *starts*.

(D7) Why departure = ∞ . ∞ is not a time, it is a sentinel guaranteeing that $\min(\text{event list})$ is an arrival. Omitting it creates a “phantom departure” for a customer who is not there.

(D8) Arrival-time and stopping-time formulas.

$$t_i^{\text{arr}} = \sum_{j=1}^i A_j, \quad T(n) = \text{clock at the event where the } n\text{-th delay completes.}$$

$T(n)$ is itself a random variable — you cannot know it before running the model.

(D9) Flow-chart facts that get tested like formulas. Both validation “No” branches (Step 3 assumptions, Step 6 model) return to **Step 2**, never to the immediately preceding step. V&V triangle edges: Real $\leftrightarrow$ Conceptual = *conceptual validation*; Conceptual $\leftrightarrow$ Operational = *verification*; Real $\leftrightarrow$ Operational = *calibration & validation*.

Quick Numbers (memorize)

The deck's single-server run (memorize the whole line). $A = 0.4, 1.2, 0.5, 1.7, 0.2, 1.6, 0.2, 1.4, 1.9$; $S = 2.0, 0.7, 0.2, 1.1, 3.7, 0.6$; stop at $n = 6$ delays.

- Arrival times 0.4, 1.6, 2.1, 3.8, 4.0, 5.6, 5.8, 7.2 (and 9.1 for C9).
- 13 events = 8 arrivals + 5 departures. $T(6) = 8.6$.
- Delays $D_1 \dots D_6 = 0, 0.8, 1.0, 0, 0.9, 3.0$; $\sum D_i = 5.7$.
- $\int Q dt = 9.9$, $\int B dt = 7.7$.
- $\hat{d}(6) = 5.7/6 = \mathbf{0.95}$; $\hat{q}(6) = 9.9/8.6 = \mathbf{1.1512}$ (slide rounds to 1.15); $\hat{u}(6) = 7.7/8.6 = \mathbf{0.8953}$ (slide rounds to 0.90 — it is *not* exactly 0.90).
- Busy intervals $[0.4, 3.3]$ (length 2.9) and $[3.8, 8.6]$ (length 4.8); $2.9 + 4.8 = 7.7$. Idle only on $[0, 0.4]$ and $[3.3, 3.8]$, total 0.9, and $8.6 - 0.9 = 7.7$ ✓

Little's Law on that very run. $\lambda_6 = 6/8.6 = 0.6977$ and $\lambda_6 \hat{d}(6) = 5.7/8.6 = \mathbf{0.6628}$, which is exactly the queue-area contributed by C1–C6. The reported $\hat{q}(6) = 1.1512$ is larger because C7 (waits $8.6 - 5.8 = 2.8$) and C8 (waits $8.6 - 7.2 = 1.4$) contribute another 4.2: $5.7 + 4.2 = 9.9$ ✓. Counting all 8 arrivals instead: $(8/8.6) \times (9.9/8) = 1.1512$ — exact.

Stability of the deck's own data. $E[A] = 7.2/8 = 0.9$ so $\lambda = 1.111$; $E[S] = 8.3/6 = 1.3833$ so $\mu = 0.7229$; $\rho = 1.537 > 1$. The example queue is *unstable* — which is exactly why $Q(t)$ is still climbing ($Q = 2$) when the run stops.

One more event, if asked to extend to $n = 7$. e_{14} : Arr C9 at 9.1 ($Q : 2 \rightarrow 3$, $\Delta t = 0.5$). e_{15} : Dep C6 at 9.2, $D_7 = 9.2 - 5.8 = 3.4$. Then $T(7) = 9.2$, $\int Q = 11.2$, $\int B = 8.3$, $\hat{d}(7) = 9.1/7 = 1.3$, $\hat{q}(7) = 1.2174$, $\hat{u}(7) = 0.9022$.

Other numbers to have ready. Trace-bug example $\hat{L}_Q = 7/16 = 0.4375$ over $T = 16$. Biased-average trap: queue = 0 for 9 h and = 10 for 1 h gives time-average **1.0**, not 5. Healthy utilization band 70–85%. Minimum-server common-sense rule: $100/12 \rightarrow 9$ servers (no simulation needed).

Exam Traps

- $\hat{u}(6)$ is $7.7/8.6 = 0.8953$, *not* 0.90 — the slide rounds. Know the exact value.
- $\lambda \hat{d}(n)$ does *not* equal $\hat{q}(n)$ on a short run unless you count customers consistently. Little's Law is asymptotic, not an event-by-event identity.
- The area rectangle uses the **old** state value and width clock – time of last event. Updating Q first is the single most common bug.
- $\hat{d}$, $\hat{q}$, $\hat{u}$ are not the same kind of average: $\hat{d}$ is discrete-time (plain mean over customers), $\hat{q}$ and $\hat{u}$ are continuous-time (area over time).
- “Verified” $\neq$ “valid.” A program can perfectly implement a conceptual model whose assumptions are wrong about reality.
- A validation failure sends you back to **Step 2**, not to Step 4 or 5.
- Delay D_i excludes service; time in system is $D_i + S_i$. $\hat{q}$ counts only those *waiting*, not the one in service.
- $\rho > 1$ does not make $\hat{u} > 1$; $\hat{u} \in [0, 1]$ always. What blows up is $Q(t)$, not $\hat{u}$.

14.2 RNG & Monte Carlo — Derived & Implicit Formulas**Derived & Implicit Formulas — not on the slides, but exam-critical**

(R1) The $1/\sqrt{n}$ law — the most exam-worthy hidden formula in this topic. *Standard statistics result (CLT); the deck only shows the error shrinking empirically.* For $\bar{Y}(n) = \frac{1}{n} \sum Y_i$ with Y_i i.i.d. and $\text{Var}(Y_i) = \sigma^2$,

$$\text{Var}(\bar{Y}(n)) = \frac{\sigma^2}{n}, \quad \text{SE}(\bar{Y}(n)) = \frac{\sigma}{\sqrt{n}}.$$

Immediate consequences (these are the MCQs):

$$n \rightarrow 4n \Rightarrow \text{error} \rightarrow \frac{1}{2} \text{ error}; \quad \text{error} \rightarrow \frac{1}{10} \text{ error} \Rightarrow n \rightarrow 100n.$$

In general, to shrink the error by a factor f , multiply n by f^2 .

95% confidence interval: $\bar{Y}(n) \pm 1.96 \sigma / \sqrt{n}$. **Sample size for a target accuracy E** (invert the SE):

$$\text{SE} \leq E \Rightarrow n \geq \left(\frac{\sigma}{E}\right)^2, \quad 95\% \text{ half-width} \leq E \Rightarrow n \geq \left(\frac{1.96 \sigma}{E}\right)^2.$$

This is also why Monte Carlo beats quadrature in high dimensions: the $1/\sqrt{n}$ rate does *not* depend on the dimension.

(R2) Variance of the Monte Carlo integration estimator. With $X \sim U(a, b)$ and $Y = (b - a)g(X)$,

$$\sigma^2 = \text{Var}(Y) = (b - a)^2 \left(E[g(X)^2] - E[g(X)]^2 \right),$$

$$E[g(X)^k] = \frac{1}{b - a} \int_a^b g(x)^k dx.$$

(R3) Hit-or-miss π : the indicator is Bernoulli. Let $H_i = \mathbf{1}\{X_i^2 + Y_i^2 \leq 1\}$. Then $H_i \sim \text{Bernoulli}(p)$ with $p = \pi/4$, so

$$E[H] = p = \frac{\pi}{4}, \quad \text{Var}(H) = p(1 - p) = \frac{\pi}{4} \left(1 - \frac{\pi}{4}\right) = 0.16855.$$

Since $\hat{\pi} = 4\bar{H}$,

$$\text{SE}(\hat{\pi}) = 4\sqrt{\frac{p(1-p)}{n}} = \frac{1.6422}{\sqrt{n}}, \quad 95\% \text{ CI: } \hat{\pi} \pm \frac{3.2187}{\sqrt{n}}.$$

(R4) $U(0, 1)$ moments and the products behind the autocorrelation test.

$$E[R] = \frac{1}{2}, \quad E[R^2] = \frac{1}{3}, \quad \text{Var}(R) = \frac{1}{12}.$$

For independent $A, B \sim U(0, 1)$ and the product $Y = AB$,

$$E[AB] = \frac{1}{4} \quad (\text{this is the } 0.25 \text{ that gets subtracted}),$$

$$E[(AB)^2] = \frac{1}{9}, \quad \text{Var}(AB) = \frac{1}{9} - \frac{1}{16} = \frac{7}{144}.$$

Adjacent products share one factor, so they are *not* independent:

$$E[AB \cdot BC] = E[A] E[B^2] E[C] = \frac{1}{12}, \quad \text{Cov} = \frac{1}{12} - \frac{1}{16} = \frac{1}{48},$$

while non-adjacent products have zero covariance. Assembling,

$$\text{Var}(\hat{\rho}_{i\ell}) = \frac{(M+1)\frac{7}{144} + 2M \cdot \frac{1}{48}}{(M+1)^2} = \frac{13M+7}{144(M+1)^2},$$

$$\sigma_{\hat{\rho}_{i\ell}} = \frac{\sqrt{13M+7}}{12(M+1)}.$$

(R5) M for the autocorrelation test — closed form (students mis-count this constantly).
 M is the largest integer with $i + (M+1)\ell \leq N$, i.e.

$$M = \left\lfloor \frac{N-i}{\ell} \right\rfloor - 1.$$

You then use $M+2$ values $R_i, R_{i+\ell}, \dots, R_{i+(M+1)\ell}$ and form $M+1$ products. Divide by $M+1$ — not by M , and not by the number of values.

(R6) LCG structural facts.

$$R_i \in \left\{0, \frac{1}{m}, \frac{2}{m}, \dots, \frac{m-1}{m}\right\} \Rightarrow \text{grid spacing} = \frac{1}{m}, \quad \max R_i = \frac{m-1}{m}, \quad R_i \neq 1 \text{ ever.}$$

- There are at most m distinct states, so the period satisfies $P \leq m$; the map $X \mapsto (aX+c) \bmod m$ is a *function*, so once any value repeats the *entire* sequence repeats from there.
- For $c = 0$ the state 0 is absorbing, so $P \leq m-1$ automatically and $X_0 = 0$ is fatal.
- The three max-period cases in the deck are the standard *Hull–Dobell*-type conditions (the deck gives the conditions but never the name):
 - (a) $m = 2^b$, $c \neq 0$: $P = m$ iff $\gcd(c, m) = 1$ and $a = 1 + 4k$.
 - (b) $m = 2^b$, $c = 0$: $P_{\max} = m/4 = 2^{b-2}$, needs X_0 odd and $a = 3 + 8k$ or $a = 5 + 8k$.
 - (c) m prime, $c = 0$: $P_{\max} = m-1$, needs the smallest k with $m \mid (a^k - 1)$ to be $k = m-1$ (i.e. a a primitive root mod m).
- **These three cases are not exhaustive.** If m is neither a power of 2 nor prime (e.g. $m = 100$), none of them applies and the period can be tiny. The general full-period rule for $c \neq 0$ is: $\gcd(c, m) = 1$; $(a-1)$ divisible by every prime factor of m ; and $(a-1)$ divisible by 4 if $4 \mid m$.

(R7) Testing formulas that are left implicit.

$$\text{chi-square: } E_i = \frac{N}{n}, \quad \text{df} = n - 1 \text{ (bins} - 1\text{)},$$

$$\chi_0^2 = \sum_{i=1}^n \frac{(O_i - E_i)^2}{E_i}.$$

Note $\sum O_i = \sum E_i = N$ always — a free arithmetic check. Equal-width bins on $[0, 1]$ each have probability $1/n$ under H_0 .

$$D^+ = \max_i \left(\frac{i}{N} - R_{(i)} \right), \quad D^- = \max_i \left(R_{(i)} - \frac{i-1}{N} \right), \quad D = \max(D^+, D^-).$$

D^+ catches data shifted *low*; D^- catches data shifted *high*. Chi-square discards within-bin position (needs $N \gtrsim 50$); K-S uses every point and works for tiny N .

$$P(\geq 1 \text{ false reject in } k \text{ tests}) = 1 - (1 - \alpha)^k, \quad E[\# \text{rejects}] = k\alpha.$$

(R8) Uniformity $\nRightarrow$ independence. The sequence 0.01, 0.02, 0.03, ... passes every uniformity test yet is perfectly predictable. Conversely a stream can be independent but biased. Both properties must be tested *separately* — that is the entire reason the autocorrelation test exists.

Quick Numbers (memorize)

Multiple-testing table ($\alpha = 0.05$) — memorize at least $k = 10$ and $k = 20$.

k (tests)	1	5	10	20	50	100
$1 - (0.95)^k$	0.05	0.226	0.401	0.642	0.923	0.994
$E[\# \text{rejects}] = 0.05k$	0.05	0.25	0.5	1.0	2.5	5.0

$U(0, 1)$ constants. $E[R] = 0.5$; $E[R^2] = 1/3$; $\text{Var}(R) = 1/12 \approx 0.0833$; $\text{SD}(R) = 1/\sqrt{12} \approx 0.2887$.

Products of two independent uniforms. $E[AB] = 1/4$; $E[(AB)^2] = 1/9$; $\text{Var}(AB) = 7/144 \approx 0.04861$; Cov of adjacent products $= 1/48 \approx 0.02083$.

Autocorrelation σ values, $\sigma = \sqrt{13M + 7}/[12(M + 1)]$: $M=2$: $\sqrt{33}/36 = 0.1596$; $M=3$: $\sqrt{46}/48 = 0.1413$; $M=4$: $\sqrt{59}/60 = 0.1280$ (deck example); $M=5$: $\sqrt{72}/72 = 0.1179$; $M=9$: $\sqrt{124}/120 = 0.0928$.

Critical values. $z_{0.025} = 1.96$. $\chi_{0.05}^2$: $\text{df}=3$: 7.81; $\text{df}=4$: 9.49; $\text{df}=5$: 11.07; $\text{df}=9$: 16.92 (deck example); $\text{df}=19$: 30.14. K-S $D_{0.05}$: $N=5$: 0.565 (deck example); $N=10$: 0.410; $N=20$: 0.294.

Deck worked answers. LCG $(X_0, a, c, m) = (27, 17, 43, 100)$ gives 2, 77, 52 ($R = 0.02, 0.77, 0.52$) — and its period is only 4: $27 \rightarrow 2 \rightarrow 77 \rightarrow 52 \rightarrow 27$. $13X \bmod 64$: seeds 1, 2, 3, 4 give periods 16, 8, 16, 4; the maximum is $m/4 = 16$. K-S example: $D = 0.26 < 0.565$, do not reject. Chi-square example: $\chi_0^2 = 3.4 < 16.9$, do not reject. Autocorrelation example: $M = 4$, $\hat{\rho} = -0.1945$, $\sigma = 0.1280$, $Z_0 = -1.52$, do not reject. $a = 7^5 = 16807$, $c = 0$, $m = 2^{31} - 1$ (prime) gives $P = m - 1$ and $\text{gap } 1/m \approx 4.66 \times 10^{-10}$.

Monte Carlo constants. $\pi/4 = 0.7854$; $p(1 - p) = 0.16855$; $\text{SE}(\hat{\pi}) = 1.6422/\sqrt{n}$. $n = 10^4$ gives $\text{SE} = 0.0164$ and 95% half-width 0.0322. Half-width ≤ 0.01 needs $n \geq 103,599$; ≤ 0.001 needs $n \geq 1.036 \times 10^7$. For $\int_0^\pi \sin x \, dx = 2$: $E[g] = 2/\pi = 0.6366$, $E[g^2] = 1/2$, $\sigma_Y = \pi\sqrt{1/2 - 4/\pi^2} = 0.9669$, so $\text{SE} = 0.9669/\sqrt{n}$ ($n=10 \Rightarrow 0.306$; $n=160 \Rightarrow 0.076$); $\text{SE} \leq 0.01$ needs $n \geq 9349$.

Exam Traps

- “Doubling n halves the Monte Carlo error” is **FALSE**. Doubling divides it by $\sqrt{2} \approx 1.41$; you must *quadruple* n to halve the error.
- Passing a uniformity test says nothing about independence. Quote 0.01, 0.02, 0.03, ... as the counterexample.
- Maximum period for a multiplicative LCG with $m = 2^b$ is $m/4$ — not m , not $m - 1$. Only the *mixed* case reaches m ; only the *prime-modulus* case gives $m - 1$.
- $m = 100$ is neither 2^b nor prime, so none of the three slide cases applies. Do not “check” $a = 1 + 4k$ and conclude full period — the deck’s own $(17, 43, 100)$ generator has period 4.
- Chi-square df is $(\text{bins} - 1) = n - 1$, *not* $N - 1$ and not the number of bins.
- M in the autocorrelation test is $\lfloor (N - i)/\ell \rfloor - 1$; you use $M + 2$ numbers, form $M + 1$ products, and divide by $M + 1$.
- The autocorrelation test is **two-tailed**: a strongly *negative* $\hat{\rho}$ is just as damning as a positive one. Compare $|Z_0|$ with 1.96.
- $\hat{\rho}_{i\ell}$ is not a correlation coefficient with the usual $[-1, 1]$ meaning; it is a mean product minus 0.25, so its natural range is $[-0.25, 0.75]$.
- Failing one test among many is expected: at $\alpha = 0.05$ with $k = 10$ tests there is a 40% chance of at least one false rejection.
- $R_i = X_i/m$ can equal 0 but never 1. Some texts divide by $m + 1$ to dodge the 0; the deck’s $2^{31} - 1$ example divides by $m + 1 = 2^{31}$.
- Monte Carlo error is itself *random*: $\bar{Y}(40)$ being worse than $\bar{Y}(20)$ in one run does not contradict $1/\sqrt{n}$, which describes a standard deviation, not any single run.
- A larger m improves *density* (gap $1/m$), not automatically the period — the period also needs the right a , c and seed.

Part III — Master Comparison Tables

Every “compare these two methods” question, answered in one place

15 Master Comparison Tables

Key Point / Exam Tip

Most “which of the following is true” MCQs are really asking you to compare two methods along one axis: *what it guarantees*, *how fast it converges*, *what it costs*, or *what it needs to start*. These tables put every comparison the course can ask about in one place.

15.1 Root-Finding Methods

Method	Type	Needs	Convergence	Key property
Bisection	Bracketing	2 guesses, $f(x_\ell)f(x_u) < 0$	Linear, rate 1/2	Always converges; iteration count known in advance; ignores how close f is to 0
False Position	Bracketing	2 guesses, sign change	Linear (generally)	Usually faster, but an endpoint can freeze; <i>can</i> be slower than bisection
Newton–Raphson	Open	1 guess + $f'(x)$	Quadratic ($p = 2$)	Fastest when it works; no convergence guarantee; 4 distinct failure modes

On the same floating-ball problem: bisection needed 10 iterations (2 digits), false position 5 (about 5 digits), Newton–Raphson 3 (digits doubling $0 \rightarrow 2 \rightarrow 4$).

15.2 Linear-System Methods

Method	End state of A	Cost (1 RHS)	When it is the right choice
Naive Gauss	Upper triangular	$\sim \frac{2}{3}n^3$	Baseline; fails on a zero pivot, fragile to round-off
Gauss + partial pivoting	Upper triangular	$\sim \frac{2}{3}n^3 + \text{swaps}$	Default. Fixes zero pivots and round-off blowup for negligible extra cost
Gauss–Jordan	Identity	$\sim n^3$	When you want the answer read straight off the RHS (no back substitution); more arithmetic
LU decomposition	$A = LU$ stored	= Gauss exactly	Many right-hand sides. Decompose once, reuse: A^{-1} costs $\sim n^3$ vs Gauss’s $\sim n^4$

15.3 Optimization Methods

Method	Derivatives	Per-iteration cost	Convergence	Main weakness
Golden Section	None	1 function eval (after the first iter.)	Linear, factor $R = 0.618$	1-D only; needs unimodality and a bracket
Gradient Descent	1st order (∇f)	$O(n)$	Linear; depends on α and $\kappa(H)$	Learning-rate tuning; slow zig-zag when ill-conditioned
Newton (opt.)	2nd order ($\nabla f, H$)	$O(n^3)$ (solve $Hy = g$)	Quadratic	Hessian cost; converges to <i>any</i> stationary point, including maxima/saddles
Lagrange multipliers	1st order + constraint	solve the system	exact (analytic)	Only locates stationary points; needs the bordered Hessian to classify

GD vs Newton, one line: use GD when n is large (the Hessian is unaffordable); use Newton when n is small (buy the quadratic convergence).

15.4 Interpolation Methods

Method	Scope	Adding a new point	Notes
Lagrange	Global	Rebuild everything	Same polynomial as Newton, different form; clean to state, expensive to extend
Newton div. diff.	Global	Append one row + one term	The “recalculation-free” property; top diagonal = coefficients
Linear spline	Local	Trivial	Continuous but kinked; no curvature
Quadratic spline	Local	—	$3n$ unknowns; matches P' at interior knots
Cubic spline	Local	—	$4n$ unknowns; matches P' and P'' ; smoothest, costliest; avoids Runge

Regression vs interpolation: regression finds a best fit and need *not* pass through any point (tolerates noise); interpolation passes through *every* point exactly (and so chases noise if the data are noisy).

15.5 RNG Statistical Tests

Test	Property tested	Statistic	Reject H_0 when
Kolmogorov–Smirnov	Uniformity	$D = \max(D^+, D^-)$	$D > D_\alpha$; works for small N , more powerful
Chi-square	Uniformity	$\chi_0^2 = \sum \frac{(O_i - E_i)^2}{E_i}$	$\chi_0^2 > \chi_{\alpha, n-1}^2$; needs large N ($\gtrsim 50$)
Autocorrelation	Independence	$Z_0 = \hat{\rho}_{i\ell} / \sigma_{\hat{\rho}_{i\ell}}$	$ Z_0 > 1.96$ at $\alpha = 0.05$ (two-tailed)

Critical reminder: uniformity tests ignore *ordering*. A perfectly uniform but perfectly predictable sequence (0.01, 0.02, 0.03, ...) sails through K-S and chi-square and is caught only by the autocorrelation test.

15.6 Convergence Orders — One Table

Method	Order p	Error behaviour
Bisection	1 (linear)	$L_n = (b - a)/2^n$; exactly one bit per iteration
Golden Section Search	1 (linear)	$L_n = R^n L_0$, $R = 0.618$
False position	1 (linear, typically)	bracket can shrink very slowly if an endpoint sticks
Gradient descent	1 (linear)	$ e_{k+1} = 1 - 2a\alpha e_k $ on a quadratic
Newton–Raphson (roots)	2 (quadratic)	$e_{k+1} \approx Ce_k^2$; digits double
Newton (optimization)	2 (quadratic)	one step exactly on a quadratic objective
Newton at a multiple root	1 (linear)	rate $1 - 1/k$ for multiplicity k
Power method	1 (linear)	error $\sim \lambda_2/\lambda_1 ^k$
Monte Carlo	—	error $\propto n^{-1/2}$ (not an iteration order)